Memory for LLM Agents: A Critical Review of Approaches, Evaluation, and Governance (2023–2026)

Prepared for the `agent-memory` project*

25 October 2026

Abstract

Large language models are stateless functions of their input, so everything an agent "remembers" is an engineering decision about what to return to the prompt, what to fold into weights or latent state, and what to keep in storage the model reaches through tools. This review surveys the approaches that emerged for this problem between 2023 and late 2026, organised by substrate (parametric, latent, token-level), function (working, episodic, semantic, procedural), and control policy (who writes, consolidates, retrieves, and forgets). It covers five mechanism families, parametric and latent memory, context-resident memory and compaction, external retrieval stores from vector indexes to temporal graphs and agent-maintained wikis, experiential and procedural memory, and learned memory-management policies, together with personalisation, multi-agent memory, evaluation, frameworks, applications, and the security, temporal-validity, provenance, and deletion literature of 2026. Unlike prior surveys, it grades every claim by evidence type and examines the field's credibility problem directly: LoCoMo scores for the same systems range from 53 to 92.5 depending on who ran them, and a plain filesystem agent has outscored several memory products. The main findings are that no mechanism family dominates and the harness often matters more than the store; recall benchmarks are saturated and poorly predict agentic use; write-time summarisation creates a barrier only raw-evidence retention can remove; similarity search cannot detect contradiction or staleness; deletion is unsolved; and memory is a compounding attack surface. The review reads the field's trajectory towards memory as a trained capability and a governed data system, sets out a research agenda with explicit success criteria, and closes with design recommendations for auditable, multi-harness memory, illustrated by a case study.

1 Introduction

1.1 Why memory, and why now

A transformer language model conditions its output on a bounded sequence of tokens. Across calls it retains nothing. Early chat applications hid this by replaying the conversation, and early "agents" extended that by replaying tool outputs. By 2023 three pressures made this untenable. Conversations and tasks outgrew the window. Users expected assistants to remember preferences and prior work across sessions. And agents began to run for hours or days, where failing to recall a constraint stated earlier (an allergy, a renamed API, a prior failed attempt) produces visible, costly errors (Zhao et al., 2026; Wu et al., 2026b).

The response has been a research and product field that now has its own surveys, benchmarks, security literature, and vendor ecosystem. Between 2024 and 2026 at least seven broad surveys appeared (Zhang et al., 2024; Du et al., 2025; Wu et al., 2025b; Hu et al., 2025; Du, 2026; Jiang

*Drafted with AI assistance (Factory Droid). Literature search conducted on 25 October 2026 using arXiv, conference proceedings, alphaXiv overviews, and vendor publications. Claims that rest on vendor-run evaluations are marked as such in the text and in the reference list.

et al., 2026; Luo et al., 2026), each proposing a taxonomy, which is itself evidence that the field has not converged on vocabulary. Hu et al. (2025), a 47-author survey, open by noting that "the proliferation of loosely defined memory terminologies has further obscured conceptual clarity" and that the long-term/short-term split "has proven insufficient."

1.2 Questions and scope

This review asks four questions:

1. What approaches exist for giving LLM agents memory, and what are the mechanisms behind each?
2. What does the evidence say about their relative effectiveness, and how trustworthy is that evidence?
3. What failure modes have been documented, including security, staleness, and deletion?
4. Where is the field heading, and which research directions does the evidence make promising?
5. What do these findings imply for the design of a memory system intended to be auditable and shared across agent harnesses?

Scope is memory for LLM-based agents: systems in which a model takes actions over multiple steps or sessions and must carry information across them. Three adjacent topics are treated only where they bear on agent memory: long-context modelling as such, retrieval-augmented generation (RAG) over static corpora, and continual pretraining. Following Hu et al. (2025), the distinction adopted here is that agent memory is writable by the agent during operation, evolves over time, and is read back to influence future behaviour. RAG over a fixed corpus and a long context window are ingredients, not memory systems.

1.3 Method and evidence grading

Sources were gathered from arXiv, conference proceedings (NeurIPS, ICML, ICLR, ACL, EMNLP, EACL, COLM), alphaXiv paper overviews, and vendor publications, with searches conducted on 25 October 2026 and citation snowballing from the surveys above. Three grades of evidence are distinguished in the text:

- **Peer-reviewed**: accepted at a venue with review (noted in the reference list where known).
- **Preprint**: arXiv only at the time of writing.
- **Vendor or practitioner report**: blog posts, READMEs, product documentation. Useful for what was shipped; weak for comparative performance.

The field moves fast and much of the strongest 2026 work is still preprint. This is noted rather than treated as disqualifying, but readers should weight claims accordingly.

1.4 Relation to prior surveys and contributions

Table 2 in Section 2 summarises the taxonomies proposed by eight surveys published between 2024 and 2026. Each is valuable as a map of mechanisms. This review differs from them in four ways.

1. **Evidence grading.** Every quantitative claim is labelled by source type (peer-reviewed, preprint, vendor), and Section 14 rates the strength of evidence for each of the field's main claims. Prior surveys report published numbers without distinguishing who produced them.
2. **The credibility problem as an object of study.** Section 10 documents the dispute over LoCoMo scores, the saturation of recall benchmarks, and the failure of recall scores to predict agentic performance, drawing on the empirical analyses of Jiang et al. (2026) and Spencer (2026).
3. **Governance as a first-class topic.** Temporal validity, provenance, belief revision, poisoning, and deletion receive a full section rather than a future-directions bullet, because

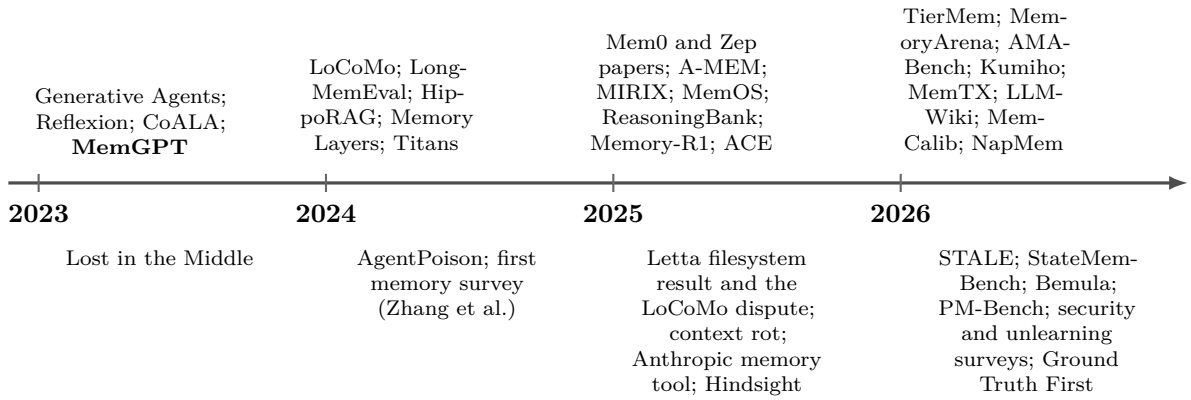


Figure 1: Timeline of the works discussed, 2023–2026. Above the line: mechanisms and systems; below: evaluation, failure modes, and governance. The field's emphasis moved from mechanisms (2023–2025) to measurement and governance (2025–2026).

the 2026 literature on these topics is now large enough, and negative enough, to change design conclusions.

4. **An actionable agenda.** Section 16 states, for each research direction, the measured gap that motivates it and what a convincing result would look like; Section 17 turns the findings into design recommendations and tests them against a concrete case.

1.5 Organisation

Section 2 fixes definitions, separates agent memory from adjacent concepts, and sets out the taxonomy. Sections 3 to 7 walk the substrate axis from inside the model outward (parametric and latent; context-resident; external retrieval) and then treat experiential memory and learned control as cross-cutting families. Section 8 covers user-centric memory, Section 9 shared memory. Section 10 covers metrics, benchmarks, and the credibility problem; Section 11 the frameworks and products; Section 12 the application domains. Section 13 covers security, temporal validity, provenance, and forgetting. Sections 14 to 16 analyse across families, read the field's trajectory, and propose a research agenda. Section 17 gives design recommendations with a case study, and Section 18 concludes. Appendix A documents the search protocol; Appendix B tabulates representative systems. Figure 1 places the works discussed on a timeline.

2 Background: Definitions, Adjacent Concepts, and Taxonomies

2.1 Definitions

An *LLM agent* is here a system in which a language model π is called repeatedly: at step t it receives a context c_t (a token sequence), emits an action a_t (text, a tool call, or an edit to its own state), and the environment and harness together produce the next context c_{t+1}. The model is stateless: $a_t = \pi(c_t)$ and nothing else. Every form of persistence therefore lives in the function that builds c_{t+1} from the history, which this review calls the *harness*, and in whatever stores that function can read.

Following Hu et al. (2025), *agent memory* is information that (i) is written during operation, by the agent, the harness, or a side process, rather than fixed before deployment; (ii) persists beyond the step or session in which it was written; (iii) evolves, in that it can be updated, consolidated, superseded, or removed; and (iv) is read back to influence later actions. A *memory system* is the combination of a store, a write policy (what is written, in what form, with what metadata), a read policy (what is retrieved and injected into c_t), a maintenance policy

Table 1: Agent memory and adjacent concepts. The last column names the condition from the definition above that the concept does not meet on its own.

Concept	What it provides	Missing on its own
Long context window	Working memory within one episode; the baseline every memory system is compared against	Persistence across sessions; degrades with length (Hong et al., 2025)
RAG over a static corpus	Read path over documents fixed before deployment	Written during operation; evolution
KV-cache reuse, prefix caching	Latent reuse of computed state across calls to the same prefix	Selective read-back; evolution; inspectability
Knowledge editing, fine-tuning	Persistent change to parameters	Selective read-back; provenance; reliable removal (Gupta et al., 2024)
Continual pretraining	Evolution of parametric knowledge from streams	Agent-written; per-item governance
Context engineering	Harness rules for assembling c_t (Anthropic, 2025a)	A store; the term names the read policy only
Agent skills, `AGENTS.md`	Human- or agent-written procedural memory loaded at start	Automatic write path; evolution during operation

(consolidation, supersession, forgetting), and the harness rules that invoke them. Much of this review's argument is that the policies and harness, not the store, determine measured behaviour.

Four quantities recur in evaluation and are used throughout. *Recall quality*: whether the information needed for a task was present in c_t when needed. *Use quality*: whether the model gave it appropriate weight (Cao et al., 2026). *Cost*: tokens, latency, and model calls across the write, store, and read paths (Omri et al., 2026). *Governability*: whether a memory's origin, validity, and status can be inspected, and whether it can be corrected or removed (Lin et al., 2026b).

2.2 Agent memory and adjacent concepts

Agent memory overlaps with several older topics, and the surveys spend effort delimiting it (Hu et al., 2025; Du et al., 2025). Table 1 records the distinctions this review uses. The short version: long context, RAG, KV-cache reuse, knowledge editing, and continual pretraining are each an *ingredient* of some memory system, but none satisfies all four conditions above on its own. A retrieval index over a fixed corpus is not written during operation; a long context window does not persist across sessions; knowledge editing persists but is not read back selectively; continual pretraining evolves but is not written by the agent.

2.3 Borrowings from cognitive science

Almost every agent memory paper borrows a human-memory frame. Three recur.

Multi-store models. Atkinson and Shiffrin's sensory, short-term, and long-term stores motivate systems such as LightMem (Fang et al., 2026), which filters input through a "sensory" stage, groups it into topic-segmented short-term memory, and consolidates into long-term memory offline. MemoryBank (Zhong et al., 2024) borrowed Ebbinghaus's forgetting curve to decay memories that are not reinforced by retrieval.

Survey	Primary axes	Notable contribution
Zhang et al. (2024)	Sources, forms, operations (writing, management, reading)	First memory-specific survey; memory as what gives agents a "self-evolving capability"
Du et al. (2025)	Representation (parametric vs. contextual) × six atomic operations: consolidation, updating, indexing, forgetting, retrieval, compression	Operation-centric view; maps operations to long-term, long-context, parametric-modification, and multi-source topics
Wu et al. (2025b)	Human memory to AI memory mapping	Cognitive grounding
Hu et al. (2025)	Forms (token-level, parametric, latent) × functions (factual, experiential, working) × dynamics (formation, evolution, retrieval)	Delimits agent memory from RAG and context engineering; compiles benchmarks and frameworks
Du (2026)	Write–manage–read loop; temporal scope × substrate × control policy; five mechanism families	Mechanism families used to structure this review
Jiang et al. (2026)	Four structures: lightweight semantic; entity-centric and personalised; episodic and reflective; structured and hierarchical	Empirical analysis of evaluation pathologies and system costs
Luo et al. (2026)	Evolutionary stages: Storage, Reflection, Experience	Historical view; ACL 2026 Findings
Lin et al. (2026b)	Six lifecycle phases × four security objectives	Security-specific; EMNLP 2026

Table 2: Taxonomies proposed by the major surveys of agent memory.

Tulving's episodic/semantic distinction, extended with procedural memory. Sumers et al. (2024) formalised this for language agents in the CoALA framework: working memory (the active context), episodic memory (records of past experience), semantic memory (facts about the world and the agent), and procedural memory (skills, including the agent's own code and prompts). This four-way split has become the lingua franca of production systems; MIRIX (Wang and Chen, 2025) adds "resource" and "knowledge vault" components, and a 2026 practitioner survey of vendors describes exactly this four-tier vocabulary as the industry default (Kumar, 2026).

Complementary learning systems. The hippocampal/neocortical fast/slow learning account motivates systems that record episodes quickly and consolidate them into abstractions slowly. HippoRAG (Gutiérrez et al., 2024) is the most explicit, modelling a knowledge graph plus Personalized PageRank as hippocampal pattern completion over a neocortical (LLM) store. Pink et al. (2025) argue, in a position paper, that episodic memory specifically is "the missing piece" for long-term agents, and define it by five properties: long-term, explicit, single-shot (acquired from one exposure), instance-specific, and contextual (bound to when and where). This definition is useful because it separates what a ledger of events provides from what a store of extracted facts provides.

These analogies are productive as design heuristics. They are weaker as evidence: no paper surveyed here shows that fidelity to a cognitive model predicts task performance, and the simplest systems in Section 5.4 ignore the analogies entirely while performing competitively.

2.4 Survey taxonomies

The surveys propose overlapping but distinct cuts. Table 2 summarises them.

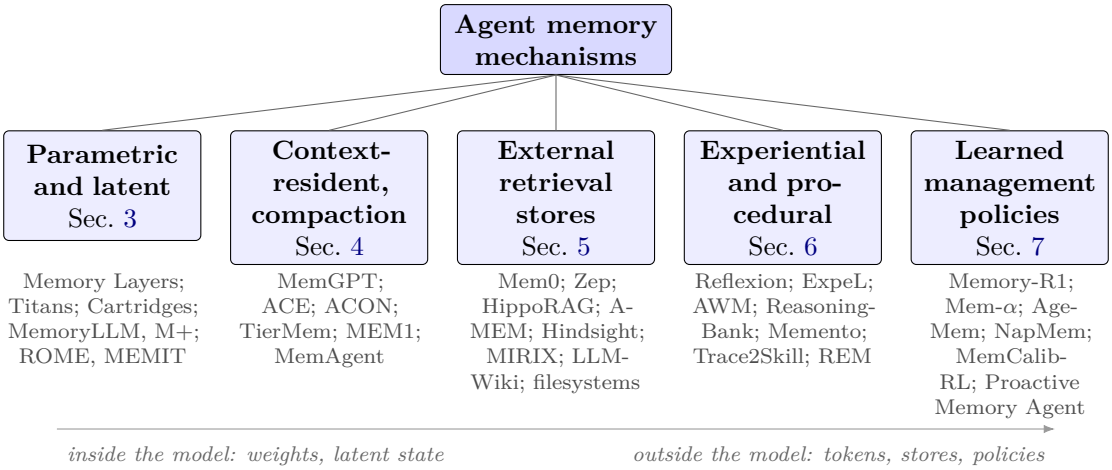


Figure 2: Taxonomy of mechanism families used in this review, ordered from inside the model outward, with representative systems. Personalisation (Section 8) and multi-agent memory (Section 9) cut across the families by memory subject; evaluation, resources, and governance are treated separately.

2.5 Working taxonomy for this review

To compare systems, this review uses three axes and one lifecycle.

Substrate. Where the information physically lives: *parametric* (model weights: fine-tuning, knowledge editing, memory layers); *latent* (continuous state that is neither weights nor tokens: KV caches, recurrent states, memory tokens, test-time-trained modules); *token-level* (text, or structured data serialised to text, that the model reads through its context window).

Function. What the information is for: working (the current task's state), episodic (what happened, when), semantic (what is true), procedural (how to act).

Control. Who decides what to write and read, and when: a fixed pipeline (an extraction LLM runs on every turn), the agent itself through tools (MemGPT's self-directed edits), a separate memory agent, a learned policy (RL-trained managers), or a human (`AGENTS.md` files).

Lifecycle. The six operations of Du et al. (2025), consolidation, updating, indexing, forgetting, retrieval, compression, plus the write-path filtering and provenance capture that the 2026 security literature adds at the front (Lin et al., 2026b).

The sections that follow walk the substrate axis from inside the model outward, then treat experiential memory and learned control as cross-cutting families, then evaluation and governance. Figure 2 shows the mechanism families treated in Sections 3 to 7, placed on the substrate axis with representative systems.

3 Parametric and Latent Memory

3.1 Extending the window from inside the architecture

The earliest "memory" work changed the model: compressed secondary memories (Rae et al., 2020), approximate k-nearest-neighbour lookup over past key-value pairs for a 262k-token effective context (Wu et al., 2022), memory tokens passed between segments (Bulatov et al., 2022), compressive memory fused with local attention (Munkhdalai et al., 2024), and explicit sparse key-value memory loaded alongside parameters (Yang et al., 2024).

These approaches extend what the model can attend to, but they inherit a known weakness of attention itself: performance degrades with input length even when the information is nominally in range. Liu et al. (2024) documented the "lost in the middle" effect. Chroma's July 2025 technical report (Hong et al., 2025) tested 18 frontier models and found that every one degraded as input length grew, including on tasks as simple as repeated-word copying; the effect is now

commonly called "context rot." RULER (Hsieh et al., 2024) and NoLiMa (Modarressi et al., 2025) showed that effective context lengths are far below advertised ones when the task requires anything beyond literal matching. These results matter for the whole field: they are why "use a bigger window" is not an answer, and why memory systems that reduce the number of tokens presented to the model can beat full-context baselines even when every needed fact is in the full context (Rasmussen et al., 2025; Chhikara et al., 2025).

3.2 Test-time learning and neural long-term memory

A second line makes part of the network learn during inference. Test-Time Training layers (Sun et al., 2024) treat the hidden state as a small model updated by gradient steps on the incoming sequence. Titans (Behrouz et al., 2024) adds a "neural long-term memory" module trained at test time with a surprise-based update rule (the gradient of the loss with respect to the input, with momentum and adaptive forgetting), reporting better scaling than Transformers and modern linear recurrent models to context windows beyond 2M tokens on needle-in-haystack tasks. Nested Learning (Behrouz et al., 2025) generalises this to a "continuum memory system" in which components update at different frequencies, and proposes the HOPE architecture as a proof of concept aimed at continual learning without catastrophic forgetting.

These are genuine memory mechanisms in the sense that they accumulate information across a stream. Their limitation for agent systems is that the memory is a tensor. It cannot be inspected, cited, corrected, exported, or shared with a different model. No agent memory survey reviewed here reports a production agent using them, and the agentic benchmarks of Section 10 do not include them.

3.3 Memory layers and latent memory tokens

Memory Layers at Scale (Berges et al., 2024) adds a trainable key-value lookup with product-key indexing, adding up to 128B memory parameters without increasing FLOPs, and shows gains on factual tasks over dense and mixture-of-experts baselines at matched compute. MemoryLLM (Wang et al., 2024b) and its extension M+ (Wang et al., 2025a) attach a pool of latent memory tokens to a frozen model and train an update mechanism; M+ adds a long-term pool with a co-trained retriever and reports retention beyond 160k tokens. Larimar (Das et al., 2024) uses an episodic memory controller for one-shot knowledge updates and selective forgetting.

A distinct and practical idea is the Cartridge (Eyuboglu et al., 2025): train a small KV cache offline on a corpus using a "self-study" recipe (the model generates questions about the corpus and trains the cache to answer them), then load that cache at inference. The authors report 38.6× memory reduction and 26.4× throughput improvement compared with placing the corpus in context, at matched quality. This is parametric memory for a document set, trained per corpus. It is relevant to agent memory as a possible consolidation target: a nightly job could compile a project's stable knowledge into a cartridge. No agent system surveyed here has yet done so.

3.4 Knowledge editing and fine-tuning

Knowledge editing methods (Meng et al., 2022, 2023) locate and modify the weights that encode a fact. They are precise for single edits but degrade under sequential editing (Gupta et al., 2024) and do not propagate consequences of an edit to related facts. Fine-tuning on interaction logs is the obvious way to make an agent "learn," but within the agent memory literature it is treated mainly as a baseline. Memento (Zhou et al., 2025a) is titled "Fine-tuning LLM Agents without Fine-tuning LLMs" precisely because the authors regard weight updates as too costly, too slow, and too opaque for continual adaptation.

3.5 Assessment

Parametric and latent memory is the only family that can make a model faster or cheaper at using what it has learned, because the information does not re-enter the context window each time. It is also the only family that offers nothing to an auditor: there is no record of what was stored, no provenance, no way to quarantine a suspect memory, and deletion means retraining or editing with uncertain side effects. Hu et al. (2025) list parametric memory as one of three forms; in practice the surveyed agent systems treat it as future work. The honest summary is that parametric memory is where the long-run efficiency gains lie and token-level memory is where everything deployable lives today.

4 Context-Resident Memory and Compaction

4.1 The window as working memory

In every agent the context window is the working memory. What distinguishes systems is how they manage it as it fills. Anthropic's September 2025 guidance (Anthropic, 2025a) names three strategies that have become standard vocabulary: *compaction* (summarise the conversation and restart with the summary), *structured note-taking* (write notes to storage outside the window and pull them back when needed), and *sub-agent architectures* (delegate a sub-task to a fresh context and return a short summary). The same month Anthropic shipped a server-side context-editing API that clears old tool results automatically and a file-based memory tool (Anthropic, 2025b), and Claude Code's "auto memory" (Anthropic, 2026) loads the first 200 lines or 25 KB of agent-written notes at the start of every session. These are vendor documents, but they describe the baseline against which research systems are now compared.

4.2 Hierarchical virtual context: MemGPT

MemGPT (Packer et al., 2023) is the founding paper of this family and the most cited design in the production literature. It treats the context window as RAM and external storage as disk. The "main context" holds system instructions, a small editable "working context," and a FIFO message queue; "external context" comprises recall storage (the full message history, searchable) and archival storage (arbitrary documents). The key move is that the LLM itself issues function calls to page information in and out, edit its working context, and search external storage, and the system raises "interrupts" (for example, a warning when the queue is nearly full) that prompt the model to act. MemGPT outperformed fixed-context baselines on a deep-memory-retrieval (DMR) benchmark and on document QA; Letta is its productised successor. Kumar (2026) observes that the vocabulary every 2026 vendor uses is MemGPT's.

The idea's durability comes from a property that later work keeps rediscovering: a capable model given simple, well-known tools and told to manage its own memory does well, and the harness's rules about what loads and when matter as much as the store (Letta, 2025; Wooders, 2026).

4.3 Compaction and its failure modes

Compaction is lossy by construction, and three papers quantify the damage.

ACE (Zhang et al., 2026b) identifies two pathologies of iteratively rewritten contexts: *brevity bias*, where summarisers drop domain-specific heuristics in favour of concise generalities, and *context collapse*, where repeated rewriting erodes detail over time. ACE's remedy is a generator/reflector/curator loop that makes incremental, itemised updates to a structured "playbook" rather than rewriting it wholesale, reporting +10.6% on agent benchmarks and +8.6% on finance tasks over strong baselines.

ACON (Kang et al., 2026) treats the compression prompt as something to optimise: it analyses cases where a compressed context led to failure while the full context succeeded, refines natural-language compression guidelines accordingly, and distils the resulting compressor into a smaller model. It reports 26 to 54% peak-token reduction while preserving task performance on AppWorld, OfficeBench, and multi-objective QA.

TierMem (Zhu et al., 2026) names the structural problem: a *write-before-query barrier*. Summaries are produced before the system knows what a future query will hinge on, so decisive constraints (their example is a peanut allergy summarised as "dietary restrictions") can be dropped and the omission is unverifiable. Their design keeps summaries as the fast path, keeps the raw log as immutable fixed-size pages with stable identifiers, routes to the raw log only when a sufficiency check fails, and writes verified findings back as new summary units linked to their raw pages. On LoCoMo this recovers nearly all of raw-only accuracy (0.851 vs. 0.873) at 54% fewer input tokens and 61% lower latency.

4.4 Learned compaction

Rather than prompt a summariser, several systems train the model to maintain a compact state. MEM1 (Zhou et al., 2026) trains an agent with RL to update a shared internal state at every turn that integrates memory and reasoning, discarding the rest, so context size stays near constant over multi-objective tasks; it reports a 3.5× memory reduction and a 3.7× performance improvement over Qwen2.5-14B-Instruct at 16-objective tasks. MemAgent (Yu et al., 2026a) trains an overwritable token-level memory for reading documents far longer than the window, with multi-conversation RL. Both are compaction policies learned end to end, and both are evaluated on tasks where the needed information is bounded and the reward is dense.

4.5 Assessment

Context-resident memory is unavoidable and well understood. The literature's consensus points are that compaction must be anchored (keep structure and raw evidence reachable), incremental (edit rather than rewrite), and verifiable (link summaries to sources), and that the harness, not the model, should own the policy for when compaction and note-taking happen. The write-before-query barrier is the clearest argument for retaining the full event log: it is the only way to make summaries correctable after the fact.

5 External Retrieval-Based Memory

This is the largest family and the one where most products live. The four structures of Jiang et al. (2026) organise it well: lightweight semantic, entity-centric, episodic and reflective, and structured and hierarchical. A fifth, filesystems and wikis, has become prominent enough since 2025 to treat separately.

5.1 Lightweight semantic stores

The simplest design embeds text units and retrieves the top-k by cosine similarity. Generative Agents (Park et al., 2023) is the canonical early example and introduced a retrieval score that combines *recency*, *importance* (an LLM-rated salience), and *relevance*, plus periodic *reflection* that synthesises higher-level observations from recent memories. These three scoring signals and the reflection step have been reused in dozens of systems since.

Mem0 (Chhikara et al., 2025) is the most widely deployed modern instance. Its pipeline has two phases: an extraction LLM reads each new message pair with a rolling summary and recent messages and proposes candidate facts; an update LLM compares each candidate against similar existing memories and chooses ADD, UPDATE, DELETE, or NOOP. The authors' own

LoCoMo results (gpt-4o-mini as answer model, LLM judge) were 66.9 for Mem0 and 68.4 for the graph variant Mem0g, against 72.9 for the full conversation in context; the pitch was cost, with Mem0 using roughly 1.8k tokens and 1.4 s p95 per query against about 26k tokens and 17 s for full context (Chhikara et al., 2025; Kumar, 2026). The paper is a preprint and its comparative numbers were disputed, as Section 10.3 discusses.

Strengths of this structure are simplicity and generality. Weaknesses are documented repeatedly: no relational structure, so multi-hop questions require the model to reassemble facts; no model of time, so stale and current facts compete with near-identical similarity; and dependence on the extraction prompt for what is remembered at all. Jiang et al. (2026) find this structure the most backbone-sensitive.

5.2 Entity-centric, graph, and temporal stores

Knowledge-graph memory addresses multi-hop and temporal questions by storing entities and typed relations.

HippoRAG (Gutiérrez et al., 2024) builds an open-information-extraction KG over passages and retrieves by Personalized PageRank seeded from query entities, reporting large gains on multi-hop QA at lower cost than iterative retrieval. HippoRAG 2 (Gutiérrez et al., 2025), subtitled "From RAG to Memory," adds deeper passage integration and online LLM use, and positions the system as non-parametric continual learning that handles factual, sense-making, and associative tasks in one framework. GraphRAG (Edge et al., 2024) adds community detection and pre-computed community summaries for global, sense-making questions; LightRAG (Guo et al., 2025) offers a cheaper dual-level variant.

Zep (Rasmussen et al., 2025) is the production system most explicit about time. Its Graphiti engine maintains a *bi-temporal* graph: each edge carries the time the fact was true in the world and the time it was recorded or invalidated in the system, so a contradiction does not delete the old edge but closes its validity interval. The graph has episode, semantic-entity, and community layers. On the DMR benchmark Zep reported 94.8% against MemGPT's 93.4%; on LongMemEval with gpt-4o it reported 71.2% against 60.2% for full context, with latency falling from 28.9 s to 2.6 s and prompt size from about 115k tokens to 1.6k. The same table shows where graph memory loses: on single-session questions about what the assistant itself said, accuracy fell from 94.6% to 80.4%, because extraction can miss the assistant's own statements (Rasmussen et al., 2025; Kumar, 2026).

AMA-Agent (Zhao et al., 2026) extends the graph idea to agent trajectories rather than dialogue, constructing a *causality graph* over states, actions, and observations with tool-augmented retrieval. Its motivation, from the AMA-Bench study, is that existing systems "fail to capture causal and objective information and rely heavily on lossy similarity-based retrieval." Kumiho (Park, 2026) goes furthest in formalisation: a property-graph memory with an Item–Revision–Tag model (each item points to an ordered sequence of immutable revisions; mutable tags such as `latest` select the current one), typed dependency edges, and URI addressing with version pinning. Its operational semantics are shown to satisfy the AGM belief-revision postulates (K*2–K*6) and Hansson's belief-base postulates. It also introduces *prospective indexing*, indexing LLM-generated implications of a memory at write time so that implicit constraints can be found later, and reports 93.3% judge accuracy on LoCoMo-Plus, with an independent reproduction by the benchmark authors in the mid-80% range.

The consistent finding is that graphs win on temporal and multi-hop questions and on explainability (edges can be cited), and cost more to build and maintain; Jiang et al. (2026) measure the ingestion latency and throughput overhead and find it is often omitted from papers.

5.3 Episodic, reflective, and hierarchical stores

A third structure keeps records of what happened and layers abstractions over them.

MemoryBank (Zhong et al., 2024) stored dialogue, daily summaries, and a user-personality profile, with Ebbinghaus-style decay. A-MEM (Xu et al., 2025) borrows the Zettelkasten method: each memory is a note with LLM-generated keywords, tags, and a contextual description; on insertion the system retrieves related notes, creates links, and may *evolve* existing notes' descriptions in light of the new one. HiMem (Zhang et al., 2026a) segments the interaction stream into episodes using topic shifts and "surprise," extracts stable "note memory" from them, links the two layers, and performs conflict-aware reconsolidation driven by retrieval feedback. LightMem (Fang et al., 2026) pushes the consolidation step offline ("sleep-time update") and reports large token and call reductions relative to Mem0-style per-turn pipelines.

Hindsight (Latimer et al., 2025) organises memory into four logical networks: a *World* network of objective facts, an *Experience* network of the agent's own first-person history, an *Opinion* network of the agent's judgements with confidence scores and timestamps, and an *Observation* network of preference-neutral entity summaries. A retrieval component (TEMPR) extracts two to five self-contained narrative facts per conversation chunk with entity resolution and temporal links, and a reasoning component (CARA) answers and updates beliefs traceably. The authors' stated motivation is that current systems "blur the line between evidence and inference." That separation of what was observed from what was inferred is the design point most relevant to auditability, and it is rare in the literature. Hindsight reports 91.4% on LongMemEval and 89.6% on LoCoMo with a large backbone, in the authors' own runs.

MIRIX (Wang and Chen, 2025) and MemOS (Li et al., 2025) are the maximalist designs. MIRIX has six memory components (core, episodic, semantic, procedural, resource, knowledge vault), each managed by a dedicated agent under a meta-manager, with active retrieval, and extends to screenshots. MemOS proposes a "MemCube" abstraction unifying plaintext, activation (KV), and parametric memory with lifecycle scheduling and governance. Both are preprints with author-run evaluations; their contribution is architectural vocabulary more than measured gains.

5.4 Filesystems, notes, and agent-maintained wikis

The most consequential empirical finding of 2025 in this family came from Letta (2025). Lacking a native way to ingest LoCoMo, they placed each conversation into a file, gave a gpt-4o-mini agent `grep`, semantic `search_files`, `open`, and `close`, constrained it to search before answering, and scored 74.0%, above Mem0's then-reported 68.4% for its best variant and above the full-context baseline. Their explanation: models post-trained on coding use filesystem tools fluently, generate their own queries, and iterate until they find the evidence, whereas purpose-built memory tools designed for single-hop retrieval are less effective. Kumar (2026) reports a related 2026 result in which BM25 over raw transcripts outperformed every dedicated memory product tested. Wooders (2026) generalises the point in an essay arguing that "memory isn't a plugin" because the harness already controls what loads, compacts, and persists.

Vendors moved the same way. Anthropic's memory tool is a directory of files the model reads and writes (Anthropic, 2025b); Claude Code's auto memory is per-repository notes the agent writes from user corrections, distinct from the human-written `CLAUDE.md`/`AGENTS.md` instruction files that act as procedural memory (Anthropic, 2026).

Karpathy (2026) articulates the strongest version. Rather than RAG over raw documents, in which "the LLM is rediscovering knowledge from scratch on every question," the agent incrementally builds and maintains a wiki of interlinked markdown pages. There are three layers: immutable raw sources (the source of truth, never modified by the LLM), the wiki (owned entirely by the LLM), and a schema file that defines conventions and workflows. Three operations: *ingest* a source (touching 10 to 15 pages), *query* (with the insight that good answers should be filed back as pages so explorations compound), and *lint* (find contradictions, stale claims, orphan pages, missing cross-references). The pattern was adopted by several teams within months (Mem0, 2026b; Akita, 2026), and Ming et al. (2026) formalise it as "retrieval as reasoning": LLM-Wiki compiles documents into structured pages with bidirectional links and

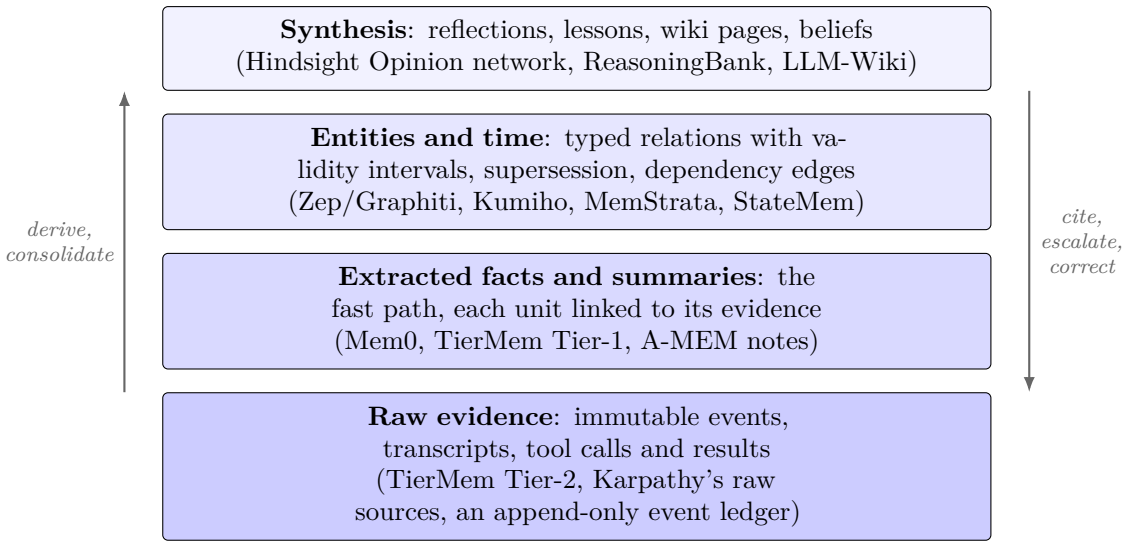


Figure 3: The layered view that the better-performing 2026 systems converge on. Higher layers are caches derived from lower ones; every unit keeps a link to the evidence it came from so that answers can be cited, summaries escalated when insufficient, and errors corrected after the fact.

source references, exposes search, read, and link-following as tools, and keeps an "Error Book" of recurring construction mistakes that constrains later compilations. It outperforms HippoRAG 2, LightRAG, and GraphRAG by 2.0 to 8.1 F1 on HotpotQA, MuSiQue, and 2WikiMultiHopQA.

Why this works deserves a careful statement, because it cuts against the research trend toward complex stores. Three reasons are supported by the evidence above. First, capability is in the model: iterative search with self-generated queries over raw or compiled text recovers from retrieval misses, while one-shot top-k cannot. Second, human-readable artifacts can be reviewed, corrected, and version-controlled with existing tools. Third, the wiki's "compile once, keep current" discipline directly attacks the staleness and contradiction problems that similarity stores cannot see (Section 13.4). The costs are equally clear: a wiki is only as good as its maintainer's lint discipline, there is no retrieval log unless one is added, and nothing in the pattern prevents a poisoned source from being compiled into a confident page.

5.5 Assessment

Across the structures the evidence supports a layered view rather than a winner (Figure 3). Raw evidence (transcripts, events) must be retained for grounding and correction; summaries and extracted facts are the fast path; entities and time-aware relations are needed when questions are temporal or multi-hop; and agent-maintained synthesis (reflections, wiki pages) is where accumulated understanding lives. The systems that perform best on the harder 2026 benchmarks combine these (Hindsight, AMA-Agent, TierMem, Kumiho, LLM-Wiki) and keep links between layers. Systems that keep only one layer, especially only extracted facts, show the documented failures: missed assistant statements, stale facts, unverifiable omissions.

6 Experiential and Procedural Memory

Luo et al. (2026) frame the field's evolution as Storage (keep trajectories), Reflection (refine them), Experience (abstract across them). This section covers the Reflection and Experience stages: memory whose content is *lessons* rather than facts.

6.1 Verbal reinforcement and insight extraction

Reflexion (Shinn et al., 2023) had an agent write a natural-language self-reflection after each failed attempt and store it in an episodic buffer that is prepended on the next attempt, improving HumanEval pass@1 to 91% and ALFWorld success substantially. ExpeL (Zhao et al., 2024) generalised from within-task to cross-task: it collects trajectories, compares successful and failed ones, and extracts "insights" that are added, upvoted, downvoted, or edited in a persistent list. Voyager (Wang et al., 2024a) built a skill library of verified executable code for Minecraft, retrieved by embedding of the skill's description, which is procedural memory in CoALA's sense.

6.2 Workflows, cheatsheets, and strategy banks

Agent Workflow Memory (Wang et al., 2024c) induces reusable sub-routines ("workflows") from past trajectories and adds them to the agent's memory, either offline from training examples or online from the agent's own successes; on WebArena and Mind2Web it improved success by 24.6% and 51.1% relative, and generalised better than baselines as the train–test gap widened. Dynamic Cheatsheet (Suzgun et al., 2025) maintains a persistent, curated memory of strategies and code snippets that the model updates at test time, more than doubling accuracy on AIME 2024 for Claude 3.5 Sonnet and giving large gains on Game of 24. ReasoningBank (Ouyang et al., 2026) distils generalisable *reasoning strategies* from both successes and failures (self-judged), retrieves them for new tasks, and adds *memory-aware test-time scaling* (MaTTS), in which parallel or sequential attempts at the same task generate richer contrast for memory synthesis; it reports up to 34.2% relative gains in effectiveness and 16% fewer steps on WebArena, Mind2Web, and SWE-Bench-Verified.

6.3 Case-based reasoning and curated contexts

Memento (Zhou et al., 2025a) frames continual adaptation as case-based reasoning over a memory-based MDP: a case bank of past (state, action, reward) triples, a learned Q-function that selects which cases to retrieve, and no update to the LLM. It reports top-1 on GAIA validation (87.9%) and strong results on DeepResearcher and SimpleQA. ACE (Section 4) belongs here as well: its playbook is procedural memory, and its contribution is a curation discipline that avoids collapse.

6.4 Skills: from lessons to reusable procedures

The newest form of experiential memory packages lessons as *skills*: named, documented procedures that a harness loads on demand, which Huang et al. (2026b) describe as an emerging ecosystem of portable, shareable agent capabilities. Human-written skills (`AGENTS.md` files, vendor skill libraries) are procedural memory authored outside the loop; the research question is whether agents can author and refine them. Trace2Skill (Ni et al., 2026) consolidates many execution trajectories in parallel, through inductive reasoning over successes and failures, into a skill directory, and can both deepen human-written skills and grow usable skills from weak LLM-generated drafts. Its evaluations span office workflows, mathematical reasoning, and visual QA; skills evolved from a 35B model's trajectories improved a 122B model by up to 57.65 percentage points on WikiTableQuestions, and the authors report that the skills outperform sequential skill editing and ReasoningBank-style retrieval memories, compress recurring failures and workarounds into standard operating procedures, and transfer across model scales, model families, and out-of-distribution settings without parameter updates or test-time retrieval. The transfer result is the important one: it is the first substantial evidence against the within-distribution caution that applies to most of this family.

REM (Chen et al., 2026) addresses experience memory for repository-level coding agents and argues that the bottleneck is lifecycle governance rather than retrieval. It makes three explicit decisions: *what to store* (each trajectory is consolidated into a phase-structured document two

orders of magnitude smaller than the trajectory), *when to recall* (the document is injected at task start, while the costlier implementation record is withheld until an in-loop gate detects that the agent is struggling), and *what to forget* (the record is evicted after one exposure). On SWE-ContextBench Lite it retrieves the related trajectory more accurately than the memory systems it compares against (68.7% vs. 55.6% match@1) and improves resolution in every replicate (27.6% vs. 24.9%) at 17% lower cost, whereas injecting the same experience as a summary or raw trajectory, even oracle-retrieved, does not help. This is a workshop paper, but it is one of very few experiential-memory results on a realistic coding benchmark with ablations over the lifecycle decisions.

6.5 Assessment

Experiential memory is where the "self-evolving agent" promise lives, and the measured gains on repeated-task distributions are real and large. Three cautions recur. First, the gains are mostly evaluated on task streams drawn from the same distribution as the experiences; transfer to new domains is weaker (Wang et al. (2024c) measure it; most do not), with Trace2Skill's cross-model and out-of-distribution results the notable exception and REM's finding that *how* experience is injected matters more than whether it is retrieved the notable complication. Second, failure-derived lessons require a judge of failure, and self-judgement is noisy; ReasoningBank uses an LLM judge and notes this. Third, a wrong lesson is a persistent wrong lesson: Du (2026) lists "trustworthy reflection" as an open challenge, and the security literature (Section 13) treats experiential stores as a prime injection target because one planted "lesson" influences every future task. Provenance for lessons (which trajectories they came from, who judged them) is almost never recorded in these systems, and the skills ecosystem makes this worse by design, since a skill is meant to travel.

7 Learned Memory-Management Policies

The newest family trains the memory controller rather than prompting it. Du (2026) calls it "policy-learned management" and Hu et al. (2025) list RL integration as a frontier.

Memory-R1 (Yan et al., 2026) trains two components: a Memory Manager that chooses ADD, UPDATE, DELETE, or NOOP for each extracted fact, and an Answer Agent that filters retrieved memories before answering, using PPO or GRPO with outcome rewards. It reports state-of-the-art results on LoCoMo from only 152 training question–answer pairs, with transfer across backbones. Mem-α (Wang et al., 2025b) trains an agent to construct a multi-component memory (core, episodic, semantic) with RL and reports generalisation from 30k-token training instances to 400k-token evaluations. AgeMem (Yu et al., 2026b) unifies long- and short-term management as tool actions (store, retrieve, update, summarise, discard) within the agent's own policy, trained by a three-stage curriculum with step-wise GRPO to handle sparse rewards. MEM1 and MemAgent (Section 4) are the context-compaction instances.

The Proactive Memory Agent (Wu et al., 2026b) is a different shape: a separate memory agent runs alongside an unmodified action agent, maintains a structured memory bank from the trajectory, and decides whether to inject a reminder or stay silent. It names the failure mode it targets, "behavioral state decay," and reports +8.3 pp pass@1 on Terminal-Bench 2.0 and +6.8 pp on τ^2-Bench, with ablations showing that *selective* intervention beats always-on injection and passive exposure of the bank. The authors also train an open-weight memory policy (Qwen3.5-27B) with SFT and GRPO, with partial transfer.

Sleep-time compute (Lin et al., 2025) is complementary: it uses idle time between interactions to pre-process context into a learned context, reporting about 5× reduction in test-time compute at matched accuracy on Stateful GSM-Symbolic and AIME, and amortisation across queries about the same context. LightMem's offline consolidation and Karpathy's lint step are the same

idea in different clothing.

Learning to read, not only to write. Two 2026 systems extend training from the write path to the read path. NapMem (Xu et al., 2026b) organises user history into a linked multi-granularity pyramid (raw conversations, typed memory records, topic tracks, user profiles) connected by provenance relations, exposes each level as a tool, and trains the agent with memory-tool RL to navigate between granularities before answering, on the argument that passive retrieval makes the model "a consumer of pre-selected evidence." It is competitive on PersonaMem-v2, LongMemEval, and LoCoMo while, on the authors' evaluation, preserving general reasoning and tool use. MemCalib (Cao et al., 2026) isolates a different problem: given a fixed set of retrieved propositions, frontier models systematically *over-use* some (importing an irrelevant preference) and *under-use* others (ignoring a binding constraint); standard post-training (GRPO, on-policy self-distillation) improves one direction while worsening the other. MemCalib-RL assigns credit separately for over- and under-use by ablating each proposition and localising its effect to response tokens, and reports balanced gains across Qwen3-8B, Ministral-3-8B, and Qwen3.5-35B-A3B that transfer to external benchmarks. Over-personalisation, measured by OP-Bench (Hu et al., 2026c), is the same over-use failure seen from the user's side (Section 8).

Experience memory inside the loop. REM (Section 6) is a learned-free but policy-explicit instance: its in-loop gate decides when to release costly experience to a struggling agent. Its ablations, which show that removing any of the store, recall, or forget decisions reduces resolution, make the general point that a memory *policy* is a first-class component whether it is learned or hand-designed.

Assessment. Learned controllers address a real weakness of prompted pipelines, which make the same kind of errors regardless of outcome. The evidence base is young: most results are on dialogue QA benchmarks with dense rewards; data-efficiency claims (152 examples) deserve replication; and the trained policy inherits the benchmark's notion of what is worth remembering. The Proactive Memory Agent is notable for evaluating on agentic benchmarks and for ablating against the obvious alternatives, and MemCalib for measuring a failure that no retrieval metric can see. Jiang et al. (2026) warn that RL-optimised compression tends to leave memory "largely unstructured and textual," which trades auditability for efficiency; NapMem's provenance-linked pyramid shows that a structured store and a learned policy are compatible, which is the combination Section 16 argues for.

8 Personalisation: User-Centric Memory

Huang et al. (2026b) make the *memory subject* a top-level axis: user-centric memory models a person (preferences, history, goals) so that an assistant can personalise, while agent-centric memory models the agent's own experience so that it can improve. Most of the systems above serve both, but personalisation has its own benchmarks, failure modes, and governance, and the consumer products that most people encounter (ChatGPT's memory, Claude's memory) are personalisation systems first.

8.1 Products and the user-controllable store

ChatGPT's memory (OpenAI, 2026) saves facts and preferences across chats, lets users view, edit, and delete individual memories or disable the feature, and, since 2025, also draws on chat history; its help documentation states that turning memory off does not delete past chats and that memories may be re-created from retained history if the feature is re-enabled, which is a concrete instance of the deletion gap (Section 13). Claude's project memory and auto

memory (Anthropic, 2026) are file-based and user-readable. These are vendor documents, but they establish the design that users now expect: a memory the user can inspect and edit in plain language, with a per-chat opt-out. No research system reviewed here evaluates the usability of memory controls, which is a gap given how much of the governance literature assumes a user who can review what is stored.

8.2 Benchmarks for personalisation

The dialogue benchmarks of Section 10 (LoCoMo, LongMemEval) are personalisation benchmarks in effect, since their questions concern a simulated user. Three 2025–2026 benchmarks target the harder parts. PersonaMem-v2 (Jiang et al., 2025) simulates 1,000 user–chatbot interactions over 300 scenarios with 20,000 preferences, most revealed *implicitly*, in 128k-token histories; frontier models reach only 37–48% accuracy on implicit personalisation, a reinforcement-fine-tuned Qwen3-4B reaches 53%, and the authors' agentic memory, a single human-readable memory of about 2k tokens that grows per user, reaches 55% with 16× fewer input tokens than the 32k-token full history. KnowMe-Bench (Wu et al., 2026a) builds on long autobiographical narratives rather than dialogues and asks evidence-linked questions spanning factual recall, subjective-state attribution, and principle-level reasoning; retrieval-augmented systems improve factual accuracy but errors persist on temporally grounded explanations and higher-level inferences, which the authors read as a need for "memory mechanisms beyond retrieval." OP-Bench (Hu et al., 2026c) measures the opposite failure, *over-personalisation*, formalised as irrelevance, repetition, and sycophancy, over 1,700 verified instances; it finds the problem widespread once memory is added, traces it to agents retrieving and over-attending to user memories when unnecessary, and proposes a lightweight filtering step (Self-ReCheck) that reduces it without losing personalisation.

8.3 Mechanisms specific to personalisation

Systems built for personalisation share a profile-plus-facts design: a compact, continuously updated user profile for the fast path, atomic facts with time for precision, and episodes for grounding. AtomMem (Yao et al., 2026) extracts high-value atomic facts, organises them into hierarchical event structures and *temporal profiles* that track evolving user attributes, and retrieves through an associative graph, reporting state-of-the-art LoCoMo results. EverMemOS (Hu et al., 2026a) implements an engram-inspired lifecycle (episodic trace formation into "MemCells" with atomic facts and time-bounded "Foresight" signals; semantic consolidation into thematic "MemScenes" that update a profile; reconstructive, scene-guided retrieval) and reports state-of-the-art results on LoCoMo and LongMemEval plus a profile study on PersonaMem-v2. Zep's user-centric temporal graph and Mem0's per-user extraction (Section 5) are the production equivalents.

Assessment. Personalisation inherits every finding of the preceding sections and adds two. First, the implicit case is unsolved: when preferences must be inferred rather than read, accuracy is near 50% for every approach tested. Second, more memory is not monotonically better: over-use, over-personalisation, and sycophancy are measured harms, and the first systems to address them do so by filtering at read time, which Sections 7 and 16 argue should become a trained, calibrated capability. The governance stakes are also highest here, because the subject of the memory is a person with rights over it.

9 Multi-Agent and Shared Memory

When several agents, or several harnesses driven by one user or one organisation, share memory, new problems appear that single-agent designs do not face: who may write, whose writes are trusted, how conflicting writes are reconciled, and how far a bad write propagates. Huang et al.

(2026b) treat multi-agent memory operations as a distinct topology, and Hu et al. (2025) list shared memory as a frontier.

9.1 Architectures

Three architectures recur. In *hierarchical shared memory*, agents write to a common structured store with levels of abstraction: G-Memory (Zhang et al., 2025b) organises multi-agent memory as a three-tier graph (insight, query, interaction) inspired by organisational memory theory, so that a team's accumulated insights sit above the specific queries and raw interactions that produced them. In *memory as a multi-agent system*, the memory itself is maintained by specialised agents: MIRIX's six managers (core, episodic, semantic, procedural, resource, knowledge vault) coordinate through a meta-manager, and the Proactive Memory Agent (Section 7) runs a memory agent alongside an unmodified action agent. In *organisational memory*, many agents and people contribute to a store that outlives any of them: Dhanyamraju and Raghav (2026) report a production deployment for enterprise coding agents that captures task-adjacent experience with contributor approval, curates it into reusable question–answer memories, gates security and privacy risks before admission, and retrieves for future agents; the effects on coding outcomes were still under evaluation at publication, but the lifecycle (capture with consent, curate, gate, retrieve) is the clearest published description of how shared memory is actually operated.

9.2 Governance of shared writes

MemTX (Li et al., 2026a) makes the governance argument directly: in shared memory "one agent's write becomes another agent's premise, and eventually a tool call with real side effects," so a write should not be a belief commit. It identifies six failure families: tool-result pollution (untrusted output written as fact), stale late writes (a slow agent overwriting a newer state), dirty reads of tentative state, semantic conflict between concurrent writers, permission laundering (a restricted fact re-emerging in an unrestricted summary), and cascading-rollback failures (retracting a belief without retracting what was derived from it). Its remedy stages writes in snapshot-isolated transactions carrying evidence, permissions, provenance, and validity; gates irreversible tool calls on in-flight belief state; and cascades repair when a belief is retracted. Two invariants are checked by property-based testing over 5.5 million protocol states. Portable Agent Memory (Ravindran, 2026) addresses the cross-vendor case with content-addressed entries in a Merkle-DAG provenance graph, capability-scoped disclosure, and an injection-resistant "rehydration" step for moving memory between GPT, Claude, Gemini, and Llama agents. Both are preprints and neither has been evaluated with multiple real harnesses under load.

9.3 Assessment

The architectures are mature enough to deploy; the governance is specified but untested. The convergent view, which the security literature shares (Section 13), is that shared memory needs transactional semantics, provenance, and trust levels per writer, because the blast radius of a bad write is every reader. The empirical gap is that no benchmark exercises multiple writers: MemoryArena and AMA-Bench are single-agent, and the multi-agent evaluations in G-Memory and MIRIX measure task success rather than conflict rate, repair latency, or propagation of a planted error. Section 16 returns to this.

10 Evaluation

10.1 Metrics

Zhang et al. (2024) distinguish *direct* evaluation, which inspects the memory itself, from *indirect* evaluation, which measures the agent's downstream performance. Almost all published results

are indirect, which is why disputes about them (Section 10.3) are hard to settle.

Indirect: answer quality. The dominant metrics are exact match or token F1 against a gold answer, and *LLM-as-judge* correctness, in which a model compares the agent's answer to the gold answer. The two can disagree by tens of points on the same outputs, since F1 penalises paraphrase and judges vary with the judge model, prompt, and version (Jiang et al., 2026). Agentic benchmarks replace answer correctness with task success (resolution rate, pass@1), which is harder to game but conflates memory with every other capability.

Indirect: efficiency. Input tokens per query, end-to-end latency, and number of model calls are reported inconsistently. Omri et al. (2026) argue for attributing cost separately to the *construction* (write), *retrieval*, and *generation* phases, since a system can look cheap on the read path while being expensive to build, and Jiang et al. (2026) find ingestion latency and throughput routinely omitted.

Direct: memory quality. Direct metrics are rarer and newer: precision and recall of extracted facts against a planted ground truth (Spencer, 2026); retrieval match@k against the trajectory that should have been recalled (Chen et al., 2026); contradiction-detection AUROC (Yadav, 2026); state-tracking accuracy (Fan et al., 2026); calibration of memory use, in which over- and under-use are scored per proposition (Cao et al., 2026); and unlearning metrics, in which forgetting is measured against a paired control that was never told the fact (Xie et al., 2026). These are the metrics that isolate what a memory system did, and their adoption is the clearest sign of methodological maturity in the 2026 literature.

Validity of the instrument. Separately from the metric, benchmarks differ in how ground truth was produced. Most generate conversations first and extract answers afterwards, with documented label errors and contamination; Spencer (2026) inverts the order so that gold answers are correct by construction. A metric is only as good as its labels.

10.2 Benchmarks

Tables 3 and 4 list the benchmarks that appear most in the literature, grouped by what they are designed to isolate. The grouping itself tells the field's story: until 2025 nearly every benchmark measured conversational recall; the 2026 additions measure validity, use, personalisation, prospective memory, deletion, and behaviour in agentic tasks.

Two shifts are visible. From recall to action: MemoryArena, AMA-Bench, and the use of Terminal-Bench test whether memory improves decisions, not whether a fact can be found, and current systems score far lower there. From storage to lifecycle: LongMemEval's knowledge-update and abstention categories, MemoryAgentBench's conflict resolution, STALE and State-MemBench's state tracking, and Bemula's forgetting test the operations that Du et al. (2025) enumerated and that most 2024 systems ignored.

10.3 The credibility problem

The comparative evidence in this field is unusually weak, and the weakness is documented by the participants themselves.

Mem0's April 2025 paper reported LoCoMo scores for competitors. Zep's rebuttal (Zep, 2025) identified three errors in how Zep had been run: both speakers ingested as the user, timestamps appended to message text rather than set in the `created_at` field, and sequential rather than parallel searches; rerun, Zep scored 75.1 against the 66.0 Mem0 had reported for it. Letta (2025) stated that the MemGPT numbers in the same paper could not have been produced without

Table 3: Benchmarks used to evaluate agent memory (part 1: recall, validity, use, and personalisation). *Size* is as reported in the source; "–" means not recorded in the sources consulted. *Isolates* names the memory property the benchmark is designed to measure separately from general capability.

Benchmark	Year, venue	What it tests	Size	Isolates	Notes
Conversational recall					
DMR (Packer et al., 2023)	2023	Recall of a fact from a prior session	–	Cross-session recall	Near-saturated (Zep 94.8%)
LoCoMo (Maharana et al., 2024)	2024, ACL	Single-hop, multi-hop, temporal, open-domain, adversarial QA over multi-session conversations	10 conversations, ∼300 turns each; 1,986 questions	Recall and temporal reasoning over dialogue	Each conversation fits in a 2026 window; category 5 lacks usable ground truth; speaker-attribution errors (Zep, 2025)
LongMemEval (Wu et al., 2025a)	2024, ICLR 2025	Information extraction, multi-session reasoning, temporal reasoning, knowledge updates, abstention	500 questions; S ∼115k tokens, M ∼500 sessions	Updates and abstention, not only recall	Assistants drop ∼30% over sustained interaction
MemoryAgentBench (Hu et al., 2026b)	2025, ICLR 2026	Accurate retrieval, test-time learning, long-range understanding, conflict resolution	Four competencies over adapted datasets	Incremental ingestion (no bulk pre-load)	All systems under 45 on the retrieval split (Kumar, 2026)
BEAM (Tavakoli et al., 2025)	2025	Recall and reasoning over very long conversations	Up to 10M tokens	Scale beyond any window	High evaluation cost
LoCoMo-Plus (Li et al., 2026b)	2026	Implicit constraints under cue–trigger semantic disconnect	–	Implicit constraints	String-match metrics shown misaligned
Validity, state, and use					
STALE (Chao et al., 2026)	2026	State resolution, premise resistance, implicit policy adaptation	400 scenarios, 1,200 queries, to 150k tokens	Implicit conflict and propagated invalidation	Best model 55.2% overall
StateMemBench (Fan et al., 2026)	2026	State tracking under supersession and dependency	–	Current vs. superseded state	Best memory system 0.149–0.233 gold rate
MemCalib (Cao et al., 2026)	2026	Appropriate weighting of each retrieved proposition	–	Over- and under-use, given fixed retrieval	Post-training skews one direction
OP-Bench (Hu et al., 2026c)	2026, EMNLP	Irrelevant, repetitive, or sycophantic use of user memory	1,700 verified instances	Over-personalisation	Widespread once memory is added
PM-Bench (Liu and Gabriel, 2026)	2026, COLM	Executing deferred intentions at the right cue during ongoing activity	Simulated 7-day week; 8 models × 8 configurations	Prospective memory	Best 65.1% F1; no strategy dominates
Personalisation					
PersonaMem-v2 (Jiang et al., 2025)	2025	Implicit preference inference from long histories	1,000 interactions, 300+ scenarios, 20k preferences, 128k tokens	Implicit personalisation	Frontier models 37–48%
KnowMe-Bench (Wu et al., 2026a)	2026	Factual recall, subjective-state attribution, principle-level reasoning over autobiographical narratives	–	Person understanding beyond retrieval	RAG improves facts, not explanations

Table 4: Benchmarks used to evaluate agent memory, continued: agentic, longitudinal, and downstream benchmarks. Columns as in Table 3.

Benchmark	Year, venue	What it tests	Size	Isolates	Notes
Agentic and longitudinal					
AMA-Bench (Zhao et al., 2026)	2026	Memory over agent trajectories (states, actions, observations, tool outputs)	Real and synthetic, any length	Trajectory memory and causal structure	Best system 57.2%
MemoryArena (He et al., 2026)	2026, ICML	Interdependent multi-session agentic tasks: web, planning, search, formal reasoning	–	Use of memory in action across sessions	Systems near-saturated on LoCoMo "perform poorly"
Ground Truth First (Spencer, 2026)	2026	As-of-date questions, trust by channel, injection probes, two horizons	∼380 questions, 15 types; 3- and 9-week histories	Label validity and tenure effects	Rankings invert between horizons
Bemula (Xie et al., 2026)	2026	Memory unlearning: storage, removal, preservation, relearning, extraction resistance	135 requests, 1,742 questions; 50 dialogues, paired controls	Deletion, separated from never-stored	No system passes >1.5% end to end
Letta Leaderboard (Letta, 2025)	2025	Model's memory-management ability	–	Model, with harness and tools fixed	Vendor-run
Terminal-Bench 2.0, τ^2-Bench, SWE-Bench Verified, SWE-ContextBench	2025–26	Downstream agentic tasks used to measure memory's effect on action	–	Outcome effect of memory	Used by ReasoningBank, Proactive Memory Agent, REM

a code path that did not exist, and that Mem0 did not respond to requests for the modified implementation. By 2026, Mem0's managed platform reported 92.5 on LoCoMo and 94.4 on LongMemEval (Mem0, 2026a), Hindsight reported 89.6, and MemOS 88.8; none of these used the same answer model, judge, or question subset as the 2025 rows, and only Zep's and Letta's numbers were produced to check someone else's (Kumar, 2026). Jiang et al. (2026) formalise the pathologies: benchmark saturation, judge sensitivity and metric validity (LoCoMo-Plus shows string matching misaligned with the construct), backbone-dependent accuracy, and omitted ingestion cost. A 2026 essay titled "The Benchmark Theatre" (Bloo-mind, 2026) argues that almost no published memory score should be compared with another.

The practical consequence is stated plainly by Letta (2025): "Comparing agent frameworks and agent memory tools is like comparing apples to oranges, as you can always mix and match frameworks and tools (and, of course, models)." A defensible comparison fixes the model, the judge, the ingestion path, and the question set, and reports token and latency cost alongside accuracy. Few papers do all four.

10.4 What good evaluation looks like

From the methodological papers (Hu et al., 2026b; Jiang et al., 2026; He et al., 2026; Zhao et al., 2026; Xie et al., 2026; Fan et al., 2026), a checklist emerges:

- Incremental ingestion, not a pre-loaded corpus, so the write path is tested.
- Interleaved memory and action, so use of memory is tested, not only recall.
- Knowledge updates and contradictions in the data, with questions whose correct answer changed, and grading that distinguishes a stale answer from other errors.
- Abstention items, so hallucinated recall is penalised.
- Forget requests with paired controls.
- Cost reported: tokens per query, ingestion latency, storage growth.
- Fixed backbone and judge across compared systems, with the judge validated against

human labels.
- Several backbones, because rankings change with the model.

11 Resources: Frameworks, Products, and Platform Features

Agent memory is unusual among research topics in that most of the systems people use are products, and several of the most cited papers are vendor papers. Table 5 lists the open-source frameworks, commercial products, platform features, and research systems with released code that appear most in the literature, characterised by the properties this review has argued matter: what structure the store has, who writes to it, whether it models time, and whether a stored unit can be traced to the raw evidence it came from. The table is a snapshot as of October 2026; the vendor rows describe shipped behaviour as documented by the vendors and should be read with the evidence grading of Section 1.3 in mind.

Three observations follow from the table. First, the write path is still dominated by per-turn extraction pipelines, which Sections 5 and 14 found to be the most expensive and the most prone to missing what the prompt did not anticipate; offline consolidation is spreading but is not yet the default. Second, bi-temporal validity is rare: Zep is the only widely deployed system that models it, which is consistent with the stale-fact failures measured in Section 13. Third, provenance to raw evidence is present in the research systems that perform best on the harder 2026 benchmarks (TierMem, Hindsight, EverMemOS, LLM-Wiki) and absent from the platform features most people use. The platform features compensate with legibility: a user or developer can read the files. The research question those two facts pose, whether legible files or provenance-linked structure scales better, is unresolved.

12 Applications

The memory functions an agent needs depend on what it does. This section describes the main application settings and the functions each stresses, with the systems and benchmarks that target them.

Personal assistants and companions. The original and largest setting: remember who the user is and what they said. It stresses semantic memory about the user, temporal validity (preferences change), and governance (the subject has rights). Section 8 covers it; the measured state is that explicit recall is near-solved, implicit inference is near 50%, and over-use is a measured harm.

Coding agents. Coding agents run for hours over a repository and return to the same codebase across sessions. They stress procedural memory (conventions, build commands, past fixes), working-memory management under compaction (a dropped constraint is a wrong edit), and prospective memory ("run the tests after the migration"). The shipped designs are file-based: `AGENTS.md` and `CLAUDE.md` as human-written procedural memory, auto memory as agent-written notes (Anthropic, 2026), and practitioner systems that index transcripts (Akita, 2026). The research results are newer: REM (Chen et al., 2026) shows that governed experience reuse improves resolution on SWE-ContextBench Lite, and Dhanyamraju and Raghav (2026) describe a deployed organisational memory for enterprise coding. Coding is also the setting in which outcome-linked evaluation is most feasible, because tests, reverts, and user corrections are observable.

Web, computer-use, and tool agents. Agents that operate interfaces stress episodic memory of trajectories and procedural memory of workflows. Agent Workflow Memory and

Table 5: Frameworks, products, and platform features. Write path: P = per-turn extraction pipeline; A = agent-directed through tools; H = human-written; O = offline consolidation. Time: B = bi-temporal validity; T = timestamps only; – = none modelled. Provenance: whether a stored unit links to the raw evidence it was derived from.

System	Kind	Store structure	Write	Time	Prov.	Evidence
Mem0, Mem0g (Chhikara et al., 2025)	OSS + hosted	Extracted facts in vector index; optional entity graph	P	T	partial	Vendor paper; disputed comparisons
Zep, Graphiti (Rasmussen et al., 2025)	OSS engine + hosted	Bi-temporal knowledge graph: episode, entity, community layers	P	B	yes (episodes)	Vendor paper
Letta, MemGPT (Packer et al., 2023; Letta, 2025)	OSS + hosted	Core blocks, recall (messages), archival; filesystem variant	A	T	yes (recall)	Paper + vendor benchmark
MemOS (Li et al., 2025)	OSS	"MemCube" unifying plaintext, KV, parametric memory with scheduling	P/O	T	partial	Preprint, author-run
LightMem (Fang et al., 2026)	Research, code	Sensory filter, topic-segmented short-term, offline long-term	O	T	partial	ICLR 2026
Hindsight (Latimer et al., 2025)	OSS (Vectorize)	Four networks: World, Experience, Opinion, Observation; TEMPR retrieval	P/O	T	yes	Vendor paper
A-MEM (Xu et al., 2025)	Research, code	Zettelkasten notes with links; note evolution	P	T	partial	NeurIPS 2025
HippoRAG 2 (Gutiérrez et al., 2025)	Research, code	Knowledge graph + passages with Personalized PageRank	P	–	yes (passages)	ICML 2025
MIRIX (Wang and Chen, 2025)	Research, code	Six components under a meta-manager; screenshots	A	T	partial	Preprint
Cognee (Cognee, 2026)	OSS + hosted	Graph + vector + relational, self-hosted	P	T	partial	Vendor docs only
EverMemOS (Hu et al., 2026a)	Research, code	MemCells to MemScenes to profile; Foresight signals	P/O	T	yes	Preprint
AtomMem (Yao et al., 2026)	Research, code	Atomic facts, event hierarchy, temporal profiles	P	T	partial	Preprint
TierMem (Zhu et al., 2026)	Research	Summaries over immutable raw pages with stable identifiers	P	T	yes	Preprint
LLM-Wiki (Ming et al., 2026); LLM Wiki pattern (Karpathy, 2026)	Research; practice	Agent-maintained Markdown wiki over raw sources, with lint	A/O	–	yes (sources)	Preprint; gist
Claude memory tool, auto memory (Anthropic, 2025b, 2026)	Platform feature	Files the agent reads and writes; auto-loaded notes	A/H	–	no	Vendor docs
ChatGPT memory (OpenAI, 2026)	Product feature	Saved memories plus chat-history reference; user view/edit/delete	P	–	no	Vendor docs

ReasoningBank (Section 6) were evaluated on WebArena and Mind2Web; the Proactive Memory Agent on Terminal-Bench and τ^2-Bench; MemoryArena and AMA-Bench (Section 10) were built to measure memory in exactly these interdependent, multi-session tasks, and current systems score far lower on them than on dialogue recall.

Deep research and long-horizon reasoning. Research agents accumulate findings over hundreds of steps and must not lose or contradict them. Memento's case bank and GAIA results (Zhou et al., 2025a), MEM1's constant-size state over multi-objective tasks, and sleep-time compute's amortisation over repeated queries about a context (Lin et al., 2025) target this setting. It stresses working-memory compaction and the write-before-query barrier most directly, since the question a finding will answer is unknown when it is recorded.

Multi-agent and enterprise systems. Teams of agents, and organisations in which many agents and people share one store, stress governance over everything else (Section 9): trust per writer, transactional writes, provenance, and permission boundaries that survive summarisation.

Embodied, multimodal, and simulation agents. Most of the field is text. Voyager's skill library for Minecraft (Wang et al., 2024a) and G-Memory's embodied tasks (Zhang et al., 2025b) are the procedural and multi-agent instances; Generative Agents (Park et al., 2023) introduced the memory stream with recency, importance, and relevance scoring that many later systems adopted, in a social-simulation setting. Multimodal memory is beginning: MIRIX ingests screenshots (Wang and Chen, 2025), and Luo et al. (2026) and Hu et al. (2025) both flag multimodal and world-model memory as frontiers. The specific challenges are that non-text evidence cannot be summarised into text without loss that the write-before-query barrier makes unverifiable, that embeddings for retrieval across modalities are weaker than text embeddings, and that the volume of raw evidence (video, screen recordings) makes raw retention expensive. None of the benchmarks in Section 10 is multimodal, so there is at present no measured state of the art.

13 Security, Temporal Validity, Provenance, and Forgetting

The 2026 literature's most important development is the recognition that memory is a governance problem. Lin et al. (2026b) characterise the threat landscape by three properties that distinguish it from prompt-level security: *persistence* (a bad write survives the session), *statefulness* (it changes future behaviour), and *propagation* (it spreads to other agents and sessions). Their Memory Lifecycle Framework crosses six phases (Write, Store, Retrieve, Execute, Share & Propagate, Forget & Rollback) with four objectives (Integrity, Confidentiality, Availability, Governance) and concludes that security "cannot be retrofitted at retrieval or execution time alone, but must be anchored in storage-time provenance, versioning, and policy-aware retention from the outset." They propose Verifiable Memory Governance as five architectural primitives for auditable, recoverable control of memory state.

13.1 Poisoning

AgentPoison (Chen et al., 2024) showed that a small number of planted entries in an agent's RAG store or memory can act as a backdoor triggered by optimised query patterns, with high success at under 0.1% poisoning rate; it assumed write access to the store. MINJA (Dong et al., 2025) removed that assumption: through ordinary user queries alone, an attacker induces the agent to write "indication prompts" into its own memory that later steer other users' sessions, reporting over 95% injection success and 70% attack success under idealised conditions. Sunil et al. (2026) replicate MINJA on EHR agents over MIMIC-III with three models and find that

realistic conditions, a store already populated with legitimate memories, "dramatically reduce attack effectiveness"; they evaluate two defences, composite trust scoring at input/output and memory sanitisation with trust-aware retrieval and temporal decay, and find that thresholds must be calibrated to avoid both rejecting everything and missing subtle attacks. The SSGM framework (Lam et al., 2026) proposes decoupling memory evolution from execution, with consistency verification, temporal decay, and access control enforced before consolidation.

The design implication accepted across this literature is that writes should carry a trust level determined by their source (user statement, agent inference, tool output, import), that tool-derived content is the main injection channel and should be quarantined pending review, and that retrieval should be able to filter on trust and status.

13.2 Confidentiality and leakage

Lam et al. (2026) names "topology-induced knowledge leakage," where sensitive context from one setting is consolidated into long-term memory and resurfaces in another; MemTX's "permission laundering" is the same failure seen from the access-control side (Li et al., 2026a). Hu et al. (2025) list trustworthiness and privacy as frontier issues. Production guidance converges on redaction before storage (API keys, credentials, personal data), scoping memory by project and user, and recording what was redacted so audits can account for gaps.

13.3 Defenses across the lifecycle

The defense literature is organised most usefully by *where in the lifecycle a mechanism acts*, because, as Lin et al. (2026b) argue, attacks exploit the sequence (seed at Write, amplify at Store, reactivate at Retrieve, spread at Share) and "no single intervention point [is] sufficient to break the full attack chain." They group defenses into *prevention* (keep untrusted content out), *containment* (limit what admitted content can do), and *recovery* (trace, roll back, delete). Table 6 maps mechanisms to phases and records how much evidence exists for each. The lifecycle organisation and the selection of mechanisms follow Lin et al. (2026b); descriptions and evidence status are taken from the primary sources.

Three observations follow. First, effort is concentrated at retrieval, where the techniques are inherited from RAG security and where several carry formal or certifiable guarantees (Xiang et al., 2026); A-MemGuard (Wei et al., 2026) is the one defense built for agent memory specifically, and its two mechanisms, consensus across reasoning paths and a separate store of distilled lessons, are themselves memory operations. But retrieval-time filtering is exactly the point at which Lin et al. (2026b) and Li et al. (2026a) argue defense arrives too late for a persistent store: a poisoned memory that is filtered today is still present tomorrow, and still propagates. Second, write-time prevention is the phase the governance literature prioritises and the one with the least quantitative evidence; the one evaluated mechanism, composite trust scoring, shows that the cost is a calibration problem, since thresholds that stop subtle attacks also reject legitimate writes (Sunil et al., 2026), and no study reports the false-quarantine rate on realistic traffic. Third, recovery is almost unstudied for dynamically updated memory: traceback has been demonstrated only for static RAG corpora (Zhang et al., 2025a), there is no published evaluation of rollback, and the only deletion evaluation finds every system failing.

Verifiable Memory Governance. Lin et al. (2026b) distil the requirements into five primitives, cast as predicates over the memory store, which are useful as a checklist for any design: *write authorization* (every entry is attributable to an authenticated source and passed an explicit authorization check before consolidation); *provenance visibility* (every entry carries queryable, lineage-complete provenance back to its originating write, surviving summarisation and merging); *principal-scoped retrieval* (retrieval returns only entries whose authorized scope includes the requesting principal); *rollbackability* (snapshots and write logs sufficient to restore any prior

Table 6: Defenses by lifecycle phase. Evidence: *evaluated* = at least one quantitative evaluation against an attack; *specified* = architecture or protocol described, with at most formal or property-based verification; *sparse* = named as a need, little published work.

Phase	Objective	Representative mechanisms	Evidence
Write	Prevention	Trust scoring of candidates from orthogonal signals (Sunil et al., 2026); expert-curated seed memory plus post-hoc human fact-checking of accumulated memories before deployment, treating memory as an alignment surface (Nguyen et al., 2025); source, version, sensitivity, and temporal metadata attached at write (Li et al., 2025); staged, snapshot-isolated belief commit with evidence and permissions (Li et al., 2026a); decoupling evolution from execution with verification before consolidation (Lam et al., 2026)	Evaluated for trust scoring (with calibration cost) and VerificAgent (OSWorld); specified for the rest
Store	Recovery substrate	Versioned snapshots and write logs; content-addressed, Merkle-linked records (Ravindran, 2026); immutable raw pages with stable identifiers (Zhu et al., 2026); W3C PROV-style lineage (Wang et al., 2026b)	Specified; Lin et al. (2026b) call store-time defenses "comparatively underdeveloped"
Retrieve	Containment	Trust-aware retrieval with temporal decay and pattern filtering (Sunil et al., 2026); isolate-then-aggregate generation with certifiable robustness (Xiang et al., 2026); cluster filtering plus model self-assessment (Zhou et al., 2025b); semantic and cluster filtering guided by entity–intent–relation extraction, then conflict-free generation (Si et al., 2025); poison detection from the model's activations (Tan et al., 2025); memory-native defense that validates by consensus across reasoning paths from related memories and stores detected failures as separate "lessons" (Wei et al., 2026)	Evaluated; the most developed phase. A-MemGuard cuts attack success by over 95% on agent memory; the rest are evaluated on static RAG corpora
Execute	Containment	Information-flow control: extracting control and data flow from the trusted query so untrusted data cannot alter program flow, with capabilities against exfiltration (Debenedetti et al., 2025); taint tracking of confidentiality and integrity labels with deterministic policy enforcement (Costa et al., 2025); runtime monitoring against user-specified Datalog policies over event graphs (Palumbo et al., 2026); privilege policies as symbolic rules over tool names and arguments (Shi et al., 2025); execution isolation between apps and system (Wu et al., 2025c); gating irreversible tool calls on belief commit state (Li et al., 2026a)	Evaluated for prompt-injection settings; specified for memory-gated execution
Share	Containment	Multi-user, multi-agent memory with asymmetric, time-evolving access encoded as bipartite user–agent–resource graphs (Rezazadeh et al., 2025); unsupervised detection of malicious agents from inter-agent message structure, without attack labels (Miao et al., 2026); capability-scoped disclosure and injection-resistant rehydration (Ravindran, 2026)	Evaluated for BlindGuard (multi-agent); specified for memory access control
Forget, rollback	Recovery	Iterative traceback of the poisoned texts responsible for an attack (Zhang et al., 2025a); snapshot and rollback primitives; descendant enumeration over provenance; paired-control deletion testing (Xie et al., 2026)	Sparse; traceback shown for static RAG only; the one end-to-end deletion evaluation is negative

state); and *verified forgetting* (after deletion, the probability that any probing query re-exposes the target from any substrate, raw logs, summaries, indexes, or copies, is bounded by a small ε). Measured against this checklist, the systems in Table 5 mostly satisfy none of the five formally; the research systems with provenance (TierMem, Kumiho, MemTX, Portable Agent Memory) satisfy the second and, by construction, much of the fourth; and Bemula's result is a direct measurement that the fifth is unmet everywhere. The checklist also makes clear why an append-only record is the natural substrate: it provides rollbackability and provenance visibility for free and makes write authorization a matter of recording the writer, while leaving verified forgetting as the one primitive it cannot provide alone.

13.4 Temporal validity, contradiction, and state tracking

The clearest negative result in the field concerns similarity search. Yadav (2026) shows on a calibrated dataset that cosine similarity distinguishes a *contradicting* fact from a *rephrased duplicate* with AUROC 0.59, near chance, because contradictions are often more similar to the original than paraphrases are. The consequence: a vector store presented with "the API is now `v2`" and "the API is `v1`" retrieves both with near-identical scores, and when forced to answer serves the stale value 15 to 40% of the time. MemStrata's remedy is a deterministic (subject, relation, object) supersession rule into a *bi-temporal ledger* (the SQL:2011 distinction between the time a fact was true and the time the system learned it), with no similarity threshold and no LLM call, reducing stale-fact errors to about zero at retrieval latency. Zep's Graphiti reached the same bi-temporal design from engineering practice (Rasmussen et al., 2025), and MemTX types contradiction-resolution heuristics as bitemporal write-time operators (Li et al., 2026a).

Two 2026 benchmarks show that explicit contradiction is the easy case. STALE (Chao et al., 2026) studies *implicit conflict*, where a later observation invalidates an earlier memory without negation: a user who mentioned a cycling commute and later mentions a broken leg. It distinguishes co-referential conflicts (a new value for the same attribute) from propagated conflicts (a change in one attribute invalidates a dependent one) and finds that even the best evaluated model achieves 55.2% overall, with models routinely accepting outdated presuppositions embedded in the user's query. StateMemBench (Fan et al., 2026) grades whether an answer reflects the current state, a superseded state, or neither, and finds dense retrieval and existing memory systems reach gold rates of 0.15 to 0.21; their StateMem method, which extracts state units with "derived from" links, marks superseded units inactive, and deterministically flags dependents for recomputation, improves this 1.6 to 1.8×. HiMem's reconsolidation and Karpathy's lint step attack staleness from the agent side.

The general lesson is that staleness is a modelling problem, not a retrieval-tuning problem. The store must know that a fact has a validity interval, that a later fact can close it, that dependent facts may be invalidated in turn, and it must keep the closed fact for audit.

13.5 Provenance and belief revision

Several 2026 systems treat provenance as a first-class structure. TierMem links every summary unit to its immutable raw pages and escalates to them when summaries are insufficient (Zhu et al., 2026). Kumiho gives every belief a URI, a revision history, provenance edges to evidence, and an immutable audit trail, and proves AGM postulate satisfaction for its revision operations (Park, 2026). Hindsight separates evidence from inference in distinct networks (Latimer et al., 2025). MemTX requires evidence, permissions, provenance, and validity on every record, and gates side effects on commit state (Li et al., 2026a). Portable Agent Memory uses content addressing and Merkle-DAG parent hashes for tamper evidence (Ravindran, 2026). Wang et al. (2026b) survey evidence tracing and execution provenance for agents and relate it to W3C PROV. These are preprints, and their benchmark scores are not the point. Their shared claim,

supported by the failure analyses in Sections 4 and 13.4, is that a memory system without provenance cannot explain, correct, or safely share what it remembers.

13.6 Forgetting and deletion

Forgetting appears in the surveys as an atomic operation (Du et al., 2025) and in early systems as decay (Zhong et al., 2024). Deletion on request is different: it is a correctness requirement with legal weight. Bemula (Xie et al., 2026) tests it directly with 135 forget requests of six types targeting 269 facts across 50 multi-session dialogues, each paired with a control version so that only systems that stored the target can score, and five suites for storage, removal, preservation of unrelated memories, relearning, and extraction resistance. Across six mainstream memory systems and three backbones, "no system passes more than 1.5% of forget requests end to end, and most pass none at all": systems either leave the information extractable under simple attacks or delete far more than requested. The authors note that one fact typically survives in several forms (raw dialogue, extracted facts, compressed notes), so deleting one copy leaves the others. Wang et al. (2026a) identify "backflow," where content deleted from explicit memory is reintroduced from the model's parameters, and Lin et al. (2026a) extend unlearning evaluation to agent settings. Deletion is, on this evidence, unsolved.

An append-only design interacts with this in a specific way. It cannot delete; it can retire, supersede, and exclude from retrieval, which satisfies "stop using this" but not "erase this." Content erasure requires an out-of-band administrative operation, and the audit log must then record that a redaction occurred without recording what was redacted. The "Forget & Rollback" phase of Lin et al. (2026b) names exactly this tension.

14 Cross-Cutting Analysis

14.1 Harness versus store

The single most consequential empirical pattern across Sections 4, 5, and 10 is that the harness matters more than the store. Letta's filesystem result, the BM25 results reported by Kumar (2026), the ablations in the Proactive Memory Agent showing that *when* to inject matters more than *what* is stored, and the vendors' convergence on file-based memory all point the same way. MemGPT's original insight, that a capable model managing simple tools does well, has held for three years. This does not mean stores are irrelevant: on temporal and multi-hop questions graphs win, and at 10M-token scale indexing is unavoidable. It means that evaluations which vary the store while holding the harness fixed, or vice versa, are the only ones that isolate anything.

14.2 Lossy versus lossless

Every system makes a lossy choice somewhere: extraction drops what the prompt did not ask for, summarisation drops detail, embeddings drop structure, decay drops the unreinforced. The write-before-query barrier (Zhu et al., 2026) explains why this is not fixable by better prompts. The systems that recover are those that keep raw evidence immutable and reachable (TierMem, Kumiho, Karpathy's raw sources layer, Hindsight's experience network) and treat everything else as a cache over it. Full-context baselines keep winning on recall benchmarks for the same reason; memory systems win on cost and on long horizons where full context is impossible or rots.

14.3 Who writes

Control policy is the least evaluated axis. Pipelines that run an extraction LLM on every turn (Mem0, Zep) are consistent and expensive and remember what the prompt was written to notice.

Agent-directed writes (MemGPT, memory tools, auto memory) are cheaper and adaptive and depend on the model's judgement of salience, which Wu et al. (2026b) show decays over long trajectories. Learned policies (Section 7) are promising and young. Human-written procedural memory (`AGENTS.md`) is the most reliable and the least scalable. No paper compares all four on the same task.

14.4 Cost

Jiang et al. (2026) find that ingestion latency and throughput are routinely omitted from papers and can dominate in production. The per-turn extraction pipelines are the most expensive; LightMem reports two orders of magnitude fewer LLM calls by consolidating offline. Sleep-time compute and nightly lint are the field's answer: do the expensive consolidation when no one is waiting.

14.5 Strength of evidence by claim

- *Memory systems reduce tokens and latency versus full context by an order of magnitude*: strong, replicated across vendors and independent runs.
- *A specific memory product beats others on recall*: weak; see Section 10.3.
- *Graph or temporal structure helps temporal and multi-hop questions*: moderate; consistent across Zep, HippoRAG, AMA-Agent, Kumiho, with mostly author-run evaluations.
- *Experiential memory improves repeated-task performance*: strong within distribution; weak out of distribution.
- *Similarity search cannot detect contradiction*: strong, with a calibrated measurement (AUROC 0.59) and two independent benchmarks showing low state-tracking accuracy.
- *Deletion is unsolved*: strong, one benchmark, negative across all systems tested.
- *Memory poisoning is practical*: strong for idealised conditions; moderate evidence that realistic stores reduce it.
- *Provenance-based governance improves outcomes*: argued, formalised, and partially verified (MemTX property tests; Kumiho's independent reproduction); not yet shown on shared agentic benchmarks.

15 Where the Field Is Heading

The preceding sections describe the field as it stands. This section reads the direction of travel from the same evidence: which ideas are gaining adherents, which are being abandoned, and where the 2026 literature places its own frontier. Each trend is stated with the evidence behind it and a note on how confident the reading is.

15.1 From storage to self-evolution

The framing of memory has shifted from "how does an agent remember" to "how does an agent improve." Luo et al. (2026) describe the evolution as Storage, Reflection, Experience. Huang et al. (2026b), a TMLR survey with over sixty authors, argue that AI research has entered a "second half" in which utility on long-horizon, user-dependent work (agentic coding, deep research, computer use) matters more than benchmark scores, and that memory is "the substrate through which agents self-evolve": short-term memory gates what is perceived, long-term memory consolidates it into reusable knowledge and skills, and the loop between them is how an agent learns from its own experience. ang Gao et al. (2026) place memory alongside models, tools, and architecture as one of the things that can evolve, and organise the literature by *what*, *when*, and *how* it evolves. The same framing underlies the "portable, shareable agent skills" ecosystem

that Huang et al. (2026b) describe, in which a lesson distilled by one agent is packaged for reuse by others.

The reading is confident: three independent surveys in 2026 converge on it. The caution is that Section 6 found experiential memory strong within distribution and weak outside it, so the field's aspiration has outrun its evidence. The next few years will test whether memory-mediated self-improvement generalises or whether it mostly overfits an agent to the tasks it has seen.

15.2 Memory management as a trained capability, including how much to trust each memory

Section 7 documented controllers trained to decide what to write and when. The frontier is moving one step further, to how memory is *used* once retrieved. NapMem (Xu et al., 2026b) exposes a provenance-linked memory pyramid (raw conversations, typed records, topic tracks, profiles) as a set of tools and trains the agent, with memory-tool RL, to navigate between granularities before answering, arguing that passive retrieval makes the model "a consumer of pre-selected evidence." MemCalib (Cao et al., 2026) isolates a problem that retrieval benchmarks cannot see: given a set of retrieved propositions, frontier models systematically *over-use* some (importing an irrelevant preference) and *under-use* others (ignoring a binding constraint), and standard post-training (GRPO, on-policy self-distillation) improves one direction while worsening the other. Their MemCalib-RL assigns credit separately for over- and under-use through per-proposition ablation and reports balanced gains that transfer to external benchmarks.

Together these mark a shift of attention from the store to the policy, and within the policy from writing to reading. The likely consequence is that "memory" and "agent" stop being separable components: the memory system supplies structure and provenance, and the model is trained to exploit both. Wooders (2026) make the same point from the vendor side under the slogan that memory "isn't a plugin." Confidence is moderate: the results are preprints on dialogue benchmarks, but they address a measured failure that no amount of better retrieval would fix.

15.3 Memory becomes a data-systems problem

The strongest 2026 designs borrow from databases and classical AI rather than from information retrieval. Bi-temporal validity (Zep, MemStrata), transactional belief commit with snapshot isolation and cascading repair (MemTX), formal belief-revision semantics over versioned items (Kumiho), dependency-propagated invalidation (StateMem), and provenance graphs with typed source relations (TierMem, Hindsight) are the vocabulary of temporal databases (Snodgrass, 1999), truth maintenance systems (Doyle, 1979), and the AGM theory of belief change (Alchourrón et al., 1985), applied to memories written by language models. NeuSymMS (Sultan et al., 2026) goes furthest, pairing neural fact extraction with a CLIPS rule engine that classifies, deduplicates, and reconciles subject–relation–value triples under explicit lifecycle rules in a relational database. Huang et al. (2026a) survey the same territory from decision theory and argue that what an agent carries between steps is a *belief* with three parts, a state, a transition across actions, and a likelihood that scores new observations against it, and that memory research has supplied the first, post-training the second, and almost nobody the third.

A parallel move treats memory as a workload. Omri et al. (2026) present the first systems characterisation of agent memory: a phase-aware profiling harness that attributes cost to construction, retrieval, and generation across ten systems, showing that external memory moves work rather than removing it (Mem0's serving time per query is under 0.1 s against about 38 s for a 360k-token long-context baseline, but only once the construction phase is excluded) and deriving recommendations on construction scheduling, capability floors, amortisation across query volume, freshness-versus-latency trade-offs, and fleet-scale management. Confidence is

high that this is where serious systems are going: it is where the measured failures (contradiction, staleness, unverifiable summaries, poisoning) all have known remedies.

15.4 Evaluation shifts from recall to behaviour, time, and tenure

Section 10 showed recall benchmarks saturating and agentic benchmarks appearing. Three further moves are visible in 2026. First, *new memory functions* are being isolated: PM-Bench (Liu and Gabriel, 2026) measures *prospective* memory, the ability to execute a deferred intention when its cue arrives while doing something else, adapting the Virtual Week paradigm from cognitive psychology; the best configuration (a GPT-5.4 agent) reaches 65.1% F1, no strategy dominates across models, and the failure is one of timely retrieval rather than knowledge. MemCalib measures use rather than recall. STALE and StateMemBench measure validity tracking. Second, *ground truth is being constructed before text*: Spencer (2026) samples a life script of facts with validity intervals, volatility classes, and source channels, renders chat and email from it, verifies every planted fact, and instantiates questions mechanically, so that gold answers are correct by construction and as-of-date questions and injection probes are possible. Third, *tenure* is being measured: in that instrument the ranking of five memory architectures inverts between a three-week and a nine-week history (a budgeted curated-map memory falls from 96% to 72% as it evicts content, while a provenance-typed graph rises to 90%), write-stage quality predicts downstream quality (weakly written facts fail 24% of the time against 2%), and injection resistance tracks whether provenance boundaries survive representation.

The direction is confident even though the individual results are preprints: every methodological critique in Section 10.3 is being answered by a benchmark designed around it. The practical consequence is that systems which lead today's short-horizon leaderboards may not lead the longitudinal ones.

15.5 Governance moves to write time; deletion becomes a system-level claim

The security and governance literature (Section 13) has converged on two positions that are now shaping designs. The first is that filtering at retrieval is too late: provenance, versioning, trust levels, and admission control must be applied when a memory is written (Lin et al., 2026b; Li et al., 2026a). The second is that forgetting must be argued at the level of the whole system. Xu et al. (2026a) introduce the notion of a *claim boundary*: a model-level unlearning claim covers parameters and checkpoints, a system-level claim covers the retrieval stores, caches, summaries, memories, tool effects, and shared state in which the target may also live, and present evidence from model evaluations does not establish removal across external state or later updates. Bemula's negative result and the backflow effect of Wang et al. (2026a) are concrete instances. The field is moving towards treating deletion as a dependency-tracing and recovery-testing problem rather than a weight-update problem, which is also the only framing in which an append-only ledger with redaction can make a defensible claim. Portability protocols (Ravindran, 2026) and the "skills" ecosystem raise the stakes, because memory that moves between vendors and agents carries its provenance, or its poison, with it.

15.6 Memory moves partly back into the model

Test-time learning (Titans, Nested Learning), trained KV caches (Cartridges), and memory layers are maturing in parallel with everything above, and Huang et al. (2026b) treat parametric state as one of two substrates rather than as future work. The plausible end state is hybrid: a token-level, auditable ledger as the source of record, with stable knowledge periodically compiled into parameters or caches for speed, and a revocation path when the ledger changes. No surveyed agent system has closed this loop (Section 16), and the auditability arguments of Section 13 will

keep the ledger primary. Confidence that parametric consolidation will appear in production agents within a few years is moderate; confidence that it will replace token-level memory is low.

15.7 Baselines commoditise, differentiation moves to the harness

Finally, the vendor record shows file-based memory, auto-loaded notes, context editing, and sleep-time consolidation becoming default features of agent platforms rather than products in themselves (Anthropic, 2025b, 2026; Letta, 2025). Combined with the harness-versus-store finding (Section 14), the implication is that a memory store will be judged less on its retrieval scores and more on what it lets a harness do: cite, correct, quarantine, supersede, explain, and delete. That is a different competition from the one the LoCoMo leaderboards describe, and it favours systems designed as governed records over systems designed as better indexes.

16 Open Problems and Promising Research Directions

On the evidence reviewed, nine problems remain unsolved: causally grounded retrieval over trajectories; continual consolidation without collapse; trustworthy reflection; learned forgetting and verified deletion; implicit state tracking; evaluation that isolates components; shared and portable memory with transactional semantics; parametric consolidation that keeps the token-level record authoritative; and multimodal and embodied memory, where there is as yet no measured state of the art (Section 12). This section argues which lines of attack the evidence makes promising, and for each states the research question, the measured gap that motivates it, and what a convincing result would look like. Table 7 summarises. The directions are grouped by the part of the memory lifecycle they address.

16.1 Memory as a governed record

Explicit temporal and dependency semantics, tested on validity benchmarks. The AUROC 0.59 result (Yadav, 2026) and the low state-tracking rates in STALE and StateMemBench are the clearest measured gaps in the field, and deterministic bi-temporal supersession already fixes the explicit cases (Yadav, 2026; Rasmussen et al., 2025). The open question is the *implicit* and *derived* cases: when an observed change invalidates conclusions that were drawn from the old value, and nothing in the text says so. Dependency edges between derived memories and their evidence (Fan et al., 2026; Park, 2026) make propagation possible, but no system has been shown to do it at scale with LLM-written edges, whose precision is unknown. A convincing result would report STALE and StateMemBench accuracy for a store that maintains dependency edges, with the precision and recall of the edges themselves measured, and an ablation that removes propagation. Truth maintenance systems (Doyle, 1979) solved this for symbolic assertions four decades ago; the research problem is doing it when the assertions and their justifications are extracted by a model that is sometimes wrong.

Belief revision with a likelihood. Huang et al. (2026a) observe that memory research has given agents a belief state and post-training has given them a transition, but almost nothing scores a new observation against existing beliefs. MemTX's staged commit and Kumiho's revision semantics are admission policies without a likelihood. The direction is to attach calibrated confidence to memories, derived from source trust, corroboration count, age, and consistency with other memories, and to update it on evidence in a principled way (AGM-style contraction and revision (Alchourrón et al., 1985) is the formal reference point). Success would be measured not by recall but by decision quality under contradiction: a benchmark in the style of StateMem where the correct action depends on which of two conflicting memories is believed, with calibration curves rather than a single accuracy.

Verifiable deletion across copies. Bemula's finding that no system deletes correctly (Xie et al., 2026), backflow from parameters (Wang et al., 2026a), and the claim-boundary framing of Xu et al. (2026a) together define the problem: a deletion request names a fact, but the fact lives in raw events, extracted memories, summaries, wiki pages, embeddings, caches, and possibly fine-tuned weights, and each copy has dependents. The promising path is dependency tracing over a provenance graph (which copies and derivatives descend from the target), followed by recovery testing (does the target's influence reappear after later writes or consolidation). An append-only design cannot erase, but it can enumerate exactly which records descend from a target, exclude them, and log the exclusion; whether that constitutes deletion is a policy question that the field has not yet framed precisely. A convincing study would run Bemula-style paired controls on a provenance-tracing store and report both immediate and delayed (post-consolidation) leakage.

16.2 Retrieval and use

Retrieval that follows structure rather than similarity. AMA-Bench's diagnosis (Zhao et al., 2026) is that agentic questions need causal and temporal structure over trajectories; LLM-Wiki (Ming et al., 2026) and NapMem (Xu et al., 2026b) show that letting the agent reason its way through a structured store beats one-shot vector retrieval; the BM25 and filesystem results show that iterative search over raw text is already strong. The open question is whether learned navigation policies transfer across stores and tasks or must be retrained per store. A convincing result would train a navigation policy on one store and benchmark pair and evaluate it, without retraining, on a different store with the same interface, on agentic benchmarks (MemoryArena, AMA-Bench) rather than dialogue QA.

Memory calibration and metamemory. MemCalib (Cao et al., 2026) shows that models mis-weight retrieved memories in both directions and that naive RL fixes one direction at the expense of the other; STALE (Chao et al., 2026) shows they do not notice when memories have expired. Both are failures of *metamemory*: knowing what one knows, how reliable it is, and whether it is still current. Promising ingredients exist separately: per-memory provenance and status exposed to the model at recall time, counterfactual credit assignment during training, and abstention when memories conflict. Nobody has combined them. The test is a joint benchmark on which a system must answer, qualify, or abstain depending on memory trust and freshness, scored by calibration and by downstream action quality.

Prospective memory and proactive injection. PM-Bench (Liu and Gabriel, 2026) shows that even strong agents execute deferred intentions at the right moment only about two thirds of the time, and the Proactive Memory Agent (Wu et al., 2026b) shows that *selective* injection beats both always-on injection and passive exposure. This is the memory function that long-running coding and operations agents most obviously lack: "run the migration after the tests pass," "tell the user before touching production," "revisit this when the upstream fix lands." The research questions are how to represent intentions with their triggering conditions (time, state, event), how to monitor for triggers cheaply across a long trajectory, and how to avoid the interruption cost of false alarms. A convincing result would improve PM-Bench F1 substantially with a trigger-monitoring harness that is also evaluated on a long-horizon agentic benchmark, with interruption rate reported.

16.3 Learning and consolidation

Consolidation that abstracts without destroying. ACE's brevity bias and context collapse (Zhang et al., 2026b) and TierMem's write-before-query barrier (Zhu et al., 2026) define the failure; incremental itemised edits, raw-evidence links, and offline lint (Karpathy, 2026; Fang et al., 2026) are the current remedies. What is missing is a measurement: no metric quantifies

how much a consolidation step has lost relative to the evidence it summarised, so there is nothing to optimise. A promising direction is to define consolidation loss directly, as the fraction of held-out questions answerable from the evidence but not from the summary, and to treat consolidation schedules (what to merge, when, how aggressively) as an optimisation problem with that loss and with token cost as the objectives, following the construction-scheduling recommendations of Omri et al. (2026).

Learned write and forget policies that preserve structure and transfer. RL-trained controllers (Section 7) learn what to keep but tend to produce unstructured text (Jiang et al., 2026) and are trained on dense QA rewards. Two refinements look promising. First, constrain the action space to structured operations with provenance (add fact with source; supersede fact; link evidence; retire), so the learned policy produces an auditable record, as NapMem does for reading. Second, derive rewards from downstream task outcomes in agentic settings rather than from answer accuracy, which the retrieval logs proposed in Section 17 make possible: every injected memory can be credited or blamed against the outcome of the action it preceded. The transfer test is the same as for retrieval policies: train on one benchmark, evaluate on another without retraining.

Closing the parametric loop. Cartridges (Eyuboglu et al., 2025), memory layers (Berges et al., 2024), and test-time learning (Behrouz et al., 2024) offer speed; the token-level ledger offers auditability. The unexplored design compiles the stable, high-trust portion of a ledger into a cache or adapter on a schedule, keeps the ledger as the source of record, and revokes or recompiles when the ledger changes. The research questions are which memories are worth compiling (frequency of retrieval, stability over time), how to detect drift between compiled and ledger state, and whether backflow (Wang et al., 2026a) can be bounded when a compiled memory is retired. A convincing result would show matched accuracy at lower cost on a long-horizon benchmark, with a measured revocation latency and leakage after revocation.

16.4 Evaluation science

Ground-truth-first, longitudinal, component-isolating benchmarks. The credibility problem (Section 10.3) is a methodological one and the remedies are known: fixed answerer and judge, incremental ingestion, interleaved action, cost accounting across construction, retrieval, and generation (Jiang et al., 2026; Hu et al., 2026b; Omri et al., 2026). Spencer (2026) adds three ingredients that should become standard: ground truth generated before text so labels are correct by construction, validity intervals and as-of-date questions so temporal reasoning is tested, and multiple horizons so tenure effects appear. The field also lacks *paired* designs, varying the store with the harness fixed and the harness with the store fixed, which are the only designs that attribute credit. A shared evaluation harness that enforces these would do more for the field than another leaderboard.

Outcome-linked evaluation on real traffic. Every benchmark above is synthetic. The alternative is to instrument deployed agents: record what was retrieved, what was injected, and what happened next, and measure whether memory changed behaviour for the better. No published system logs retrieval in a form that makes this possible (Section 17). A promising contribution would be an open instrument for retrieval logging with a privacy-preserving export, and a first longitudinal study of memory utility in coding agents, where outcomes (tests passing, reverts, user corrections) are observable.

16.5 Security, sharing, and portability

Write-time admission with calibrated trust. The attack literature (Chen et al., 2024; Dong et al., 2025) and the lifecycle survey (Lin et al., 2026b) agree that retrieval-time filtering is insufficient, yet Table 6 shows that retrieval is where nearly all evaluated defenses sit; MemTX formalises admission as a transaction. The gaps are empirical: trust-aware retrieval has been evaluated on idealised stores (Sunil et al., 2026), and admission policies have been verified by property tests but not shown on shared agentic benchmarks. The direction is an end-to-end evaluation in which a provenance-typed store with admission control faces adaptive poisoning across sessions, with task success, attack success, and quarantine false-positive rate reported together. Spencer (2026)'s finding that injection resistance tracks whether provenance survives representation is the hypothesis to test.

Shared memory for teams of agents and people. MemTX's six failure families and Portable Agent Memory's rehydration step describe the problem; neither has been evaluated with multiple real harnesses writing to one store under realistic load. Questions include per-writer trust that adapts to a writer's track record, conflict resolution when two harnesses supersede the same key, and disclosure control when memory crosses organisational or vendor boundaries. A multi-harness ledger (Section 17) is a natural testbed, and the measurements that matter are conflict rate, repair latency, and whether a bad write from one harness ever reaches an irreversible action in another.

16.6 Theory and cognitive-science transfer

Formalising the write-before-query barrier. TierMem names the barrier; nobody has formalised it. A rate–distortion treatment, in which the distortion is defined over a distribution of future queries and the rate is the summary's token budget, would say when lossy compaction can be safe (query distribution known and narrow) and when raw retention is necessary (query distribution broad or shifting), and would give consolidation a principled objective. The consolidation-loss metric proposed above is the empirical counterpart.

Cognitive science as benchmark design, not metaphor. The field has borrowed cognitive vocabulary freely (Section 2) but has only recently borrowed cognitive *methods*: PM-Bench adapts Virtual Week, and the sleep-time and lint work rediscovers consolidation. Other paradigms with established experimental designs translate directly: reconsolidation (a retrieved memory becomes labile and can be updated, which is what supersession-on-recall would be), schema effects (what is remembered depends on existing structure, which bears on extraction prompts), interference (similar memories compete, which bears on dedupe), and metamemory judgements (feeling-of-knowing, which bears on calibration and abstention). Each would yield a benchmark with a human baseline and a decades-old literature on what the results mean.

17 Design Recommendations for Auditable Memory, with a Case Study

This section turns the review's findings into design recommendations for anyone building a memory system that must be inspectable, correctable, and shared across more than one agent harness. Each recommendation names the evidence behind it. The second half tests the recommendations against a concrete design: a local, append-only ledger to which several coding-agent harnesses write.

Table 7: Research agenda: direction, the measured gap that motivates it, and what a convincing result would show.

Direction	Motivating evidence	Convincing result
Temporal and dependency semantics	AUROC 0.59 for contradiction; STALE 55%; StateMem 0.15–0.36	Dependency-propagating store evaluated on STALE and StateMem with edge precision reported
Belief revision with likelihood	Belief surveys: state and transition exist, likelihood absent	Calibration curves on decision-under-contradiction tasks
Verifiable deletion	Bemula $\leq$1.5%; backflow; claim boundaries	Paired-control leakage, immediate and post-consolidation, on a provenance-tracing store
Structured retrieval policies	AMA-Bench; LLM-Wiki; NapMem; BM25 and filesystem results	Navigation policy transfers across stores on agentic benchmarks
Calibration and metamemory	MemCalib over/under-use; STALE	Answer/qualify/abstain benchmark scored by calibration and action quality
Prospective memory	PM-Bench 65.1% F1; selective injection wins	Trigger-monitoring harness improves PM-Bench with interruption rate reported
Consolidation loss	Brevity bias, collapse, write-before-query	Defined loss metric; schedules optimised against it and token cost
Structured learned policies	RL output unstructured; dense QA rewards	Provenance-preserving action space; outcome-derived rewards; cross-benchmark transfer
Parametric loop	Cartridges, memory layers unused in agents	Matched accuracy at lower cost with measured revocation latency and leakage
Evaluation harness	Credibility problem; tenure crossover	Ground-truth-first, multi-horizon, paired store/harness designs with cost
Outcome-linked evaluation	No system logs retrieval	Longitudinal study on coding agents using retrieval and outcome logs
Write-time admission	Poisoning practical; retrieval filtering late	Adaptive poisoning vs. admission control with task, attack, and false-positive rates
Shared memory	MemTX failure families untested at scale	Multi-harness testbed reporting conflict rate and repair latency
Theory	Barrier named, not formalised	Rate–distortion account of compaction over query distributions

17.1 Recommendations

Keep the raw record; it is the layer the literature says cannot be reconstructed. The write-before-query barrier (Section 4), the Letta and BM25 results (Section 5.4), TierMem, and the tenure crossover of Spencer (2026) all argue that an immutable record of what happened, with tool calls and results linked, is the foundation. Extracted memories, summaries, and wiki pages are caches that must link back to it. Make the raw record a first-class recall path (full-text search over events), not only a provenance pointer, because iterative search over raw text is the most robust retrieval behaviour measured.

Admit writes; do not treat a write as a belief. MINJA and AgentPoison make tool output and query-induced writes the main poisoning channels; trust-aware retrieval is the defence Sunil et al. (2026) evaluate; MemTX's refinement is that a write should not become an actionable belief until admitted, and irreversible actions should be gated on belief state. Record a trust level per writer and per source, quarantine tool-derived memories by default, and expose admission status to the reader so an agent can see that a memory is unreviewed. This is the write-authorization primitive of Verifiable Memory Governance (Section 13.3).

Model temporal validity explicitly; do not infer it from similarity. The AUROC 0.59 result means dedupe and contradiction detection by embedding similarity will not distinguish a correction from a paraphrase. Use a bi-temporal record (validity interval in world time, recording time in system time) with a supersession link that closes the earlier interval. Deterministic supersession on a (scope, key) pair, as MemStrata does on (subject, relation), should be primary; dependency flags for derived memories, as in StateMem, propagate invalidation; similarity is a secondary signal for review queues.

Separate evidence from inference. Hindsight's four networks and Kumiho's belief graph distinguish what was observed from what was concluded. Type the relation between a memory and its sources (stated by the user, derived from evidence, supported by, contradicted by), and surface the type wherever memory is rendered, so a reader can see whether a statement is an observation or an inference.

Log retrieval, not only storage. No surveyed system records what was retrieved, with what scores, and whether it was injected, which is why "why did the agent say that" is unanswerable in most of them. Record retrievals, ranked items, and what entered the context, and make it possible to ask what a memory's status was at recall time and what it is now.

Consolidate offline. LightMem, sleep-time compute, the wiki lint step, and the construction-scheduling recommendations of Omri et al. (2026) all move expensive synthesis out of the interaction loop. Quality work (dedupe, contradiction review, projection into human-readable form) belongs in a scheduled job over an export rather than in per-turn model calls.

Plan for the deletion gap honestly. No system deletes correctly today (Xie et al., 2026), and a system-level deletion claim must cover every copy and derivative (Xu et al., 2026a). Provide an administrative redaction path that enumerates descendants through the provenance graph, logs the redaction as an event with counts rather than content, and is tested with paired controls.

Evaluate on your own traffic. Given the credibility problem, external benchmark scores for extraction or recall components will not be comparable. A system that logs retrieval and outcomes enables the evaluation the methodological papers ask for: incremental ingestion is the

capture path, interleaved action is the agents' real work, and injected memories joined to later outcomes measure whether memory changed behaviour.

17.2 Case study: an append-only, multi-harness ledger

The recommendations were developed alongside a concrete design, which serves here as a check that they are implementable together. The design is a single local analytical database, owned by the developer, to which several coding-agent harnesses write through one small API. Its storage has four layers that correspond to the recommendations above.

Events are the raw record: every session, message, tool call, and tool result, append-only, with large content in blob storage and full-text search over the text. *Memories* are curated units with a scope and key, a trust level set from the writer and source, a validity interval and recording time, and a supersession link; a derived status view reports whether a memory is active, superseded, expired, or quarantined, and tool-sourced memories are quarantined until admitted. *Sources* link each memory to the events or memories it came from with a typed relation (user-stated, derived-from, supports, contradicts), so evidence and inference are distinguishable and a redaction can enumerate descendants. *Retrievals* record every recall: the query, the ranked candidates with scores, which were injected, and a link from each injected item to the memory and its status at that moment, so that a later "why" question can be answered by a join rather than a guess.

Deduplication and contradiction review run as an offline job over an export, as does projection into a Markdown wiki in which every page states its source relations. Deletion is implemented as redaction: an event records that content was removed and how many records it touched, and the removed records are excluded from every view. The open items in the design match the open items in the literature. Dependency-propagated invalidation of derived memories is specified but not yet implemented; the quality of model-written source links has not been measured; and whether redaction-by-exclusion satisfies a deletion request is a policy question the design surfaces rather than settles. The point of the case study is not that the design is right but that the recommendations are jointly satisfiable in a small system, and that the resulting record is itself the instrument the field needs for outcome-linked evaluation.

18 Conclusion

Memory for LLM agents has gone, in three years, from a workaround for short context windows to a field with its own surveys, benchmarks, products, attack literature, and governance proposals. This review organised that field by substrate, function, and control policy, examined five mechanism families and the cross-cutting topics of personalisation and shared memory, and graded the evidence behind each claim.

The findings are not what the leaderboards suggest. No mechanism family dominates, and the harness that decides what loads, compacts, and persists often matters more than the store; a plain filesystem agent and a BM25 index have outscored dedicated products. Recall benchmarks are saturated and poorly predict agentic use of memory, where current systems score far lower, and reported scores on the same benchmark vary by forty points with the evaluation setup. Write-time summarisation creates a barrier that only retention of raw evidence with provenance can remove. Similarity search cannot detect contradiction or staleness, so temporal validity and dependency must be modelled explicitly. Deletion on request is essentially unsolved. And memory is a persistent attack surface that compounds across sessions and agents, which is why the 2026 literature argues for governance at write time.

The field's direction follows from these findings. Memory is becoming a trained capability, with policies learned for writing, reading, and weighting memories; a governed data system, borrowing bi-temporal records, transactions, provenance, and belief revision from databases and

classical AI; and the substrate through which agents are meant to improve from experience, a promise that is real within distribution and largely untested outside it. Evaluation is shifting from recall to behaviour, time, and tenure. The research agenda in Section 16 names, for each of these, the measured gap and the result that would close it.

For builders, the recommendations are concrete: keep the raw record, admit writes rather than trusting them, model time explicitly, separate evidence from inference, log retrieval, consolidate offline, plan for deletion honestly, and evaluate on real traffic. For researchers, the most valuable contributions in the near term are not new stores but better instruments: ground-truth-first longitudinal benchmarks, component-isolating protocols, direct metrics for memory quality, and outcome-linked evaluation in deployment. The systems that will matter are those designed as governed records of what an agent learned, from where, and with what confidence, rather than as better indexes over text.

Limitations and acknowledgements

This review was drafted with AI assistance and is based on abstracts, full texts where accessible, alphaXiv overviews, and vendor publications retrieved on 25 October 2026. Author lists for long author lists are abbreviated in the bibliography. Headline figures for Mem0, Zep, and Letta were cross-checked against the primary publications and an October 2026 practitioner compilation (Kumar, 2026); other reported numbers are taken from the cited papers' abstracts and have not been independently reproduced. Several of the strongest 2026 results are preprints and may change on revision.

References

Fabio Akita. ai-memory: Long-term memory for agent coding CLIs. GitHub repository and research notes, 2026, 2026. Practitioner. https://github.com/akitaonrails/ai-memory.

Carlos E. Alchourrón, Peter Gärdenfors, and David Makinson. On the logic of theory change: Partial meet contraction and revision functions. *Journal of Symbolic Logic*, 50(2):510–530, 1985.

Huan ang Gao, Jiayi Geng, Wenyue Hua, Mengkang Hu, Xinzhe Juan, Hongzhang Liu, et al. A survey of self-evolving agents: What, when, how, and where to evolve on the path to artificial super intelligence. *Transactions on Machine Learning Research*, 2026. arXiv:2507.21046.

Anthropic. Effective context engineering for AI agents. Anthropic engineering blog, 29 September 2025, 2025a. Vendor publication. https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents.

Anthropic. Managing context on the Claude developer platform: Context editing and the memory tool. Product announcement and documentation, 29 September 2025, 2025b. Vendor publication. https://www.anthropic.com/news/context-management.

Anthropic. How Claude remembers your project: CLAUDE.md, AGENTS.md, and auto memory. Claude Code documentation, 2026. Vendor publication, accessed 25 October 2026. https://code.claude.com/docs/en/memory.

Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to memorize at test time. *arXiv preprint arXiv:2501.00663*, 2024.

Ali Behrouz, Meisam Razaviyayn, Peilin Zhong, and Vahab Mirrokni. Nested learning: The illusion of deep learning architectures. In *NeurIPS*, 2025. arXiv:2512.24695.

Vincent-Pierre Berges, Barlas Oğuz, Daniel Haziza, Wen tau Yih, Luke Zettlemoyer, and Gargi Ghosh. Memory layers at scale. *arXiv preprint arXiv:2412.09764*, 2024.

Bloo-mind. The benchmark theatre: Why almost nothing you've read about agent memory

scores is comparable. Essay, 20 May 2026, 2026. Practitioner analysis. https://essays.bloo-mind.ai/posts/2026-05-20-mem-eval/.

Aydar Bulatov, Yuri Kuratov, and Mikhail Burtsev. Recurrent memory transformer. In *NeurIPS*, 2022.

Ruike Cao, Fanyu Zhao, Fugen Yao, Liang Dong, Jian Xu, Yifei Zhao, Han Zhang, et al. MemCalib: Benchmarking and optimizing memory use in LLM agents. *arXiv preprint arXiv:2609.24259*, 2026.

Hanxiang Chao, Yihan Bai, Rui Sheng, Tianle Li, and Yushi Sun. STALE: Can LLM agents know when their memories are no longer valid? *arXiv preprint arXiv:2605.06527*, 2026.

Xinyi Lisa Chen, Pedro Herrero-Vidal, Xiaohan Chen, Xiaojing Yu, Cris Liu, Bingqing Song, et al. REM: What to store, when to recall, what to forget in experience memory for coding agents. In *NeurIPS 2026 Workshop on Continual Learning for Enterprise AI Agents*, 2026. arXiv:2610.27377.

Zhaorun Chen, Zhen Xiang, Chaowei Xiao, Dawn Song, and Bo Li. AgentPoison: Red-teaming LLM agents via poisoning memory or knowledge bases. In *NeurIPS*, 2024.

Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready AI agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025. Vendor research.

Cognee. Cognee: Open-source agent memory platform. GitHub repository, https://github.com/topoteretes/cognee, 2026. Vendor publication, accessed 25 October 2026.

Manuel Costa, Boris Köpf, Aashish Kolluri, Andrew Paverd, Mark Russinovich, Ahmed Salem, et al. Securing AI agents with information-flow control. *arXiv preprint arXiv:2505.23643*, 2025.

Payel Das, Subhajit Chaudhury, Elliot Nelson, Igor Melnyk, Sarath Swaminathan, Sihui Dai, et al. Larimar: Large language models with episodic memory control. In *ICML*, 2024.

Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, et al. Defeating prompt injections by design. *arXiv preprint arXiv:2503.18813*, 2025.

Harsh Rao Dhanyamraju and Leonidas Raghav. Shared organizational memory for enterprise coding agents: System design and deployment snapshot. *arXiv preprint arXiv:2608.00122*, 2026.

Shen Dong, Shaochen Xu, Pengfei He, Yige Li, Jiliang Tang, Tianming Liu, Hui Liu, and Zhen Xiang. Memory injection attacks on LLM agents via query-only interaction. *arXiv preprint arXiv:2503.03704*, 2025. Earlier title: A Practical Memory Injection Attack against LLM Agents.

Jon Doyle. A truth maintenance system. *Artificial Intelligence*, 12(3):231–272, 1979.

Pengfei Du. Memory for autonomous LLM agents: Mechanisms, evaluation, and emerging frontiers. *arXiv preprint arXiv:2603.07670*, 2026.

Yiming Du, Wenyu Huang, Danna Zheng, Zhaowei Wang, Sebastien Montella, Mirella Lapata, Kam-Fai Wong, and Jeff Z. Pan. Rethinking memory in LLM based agents: Representations, operations, and emerging topics. *arXiv preprint arXiv:2505.00675*, 2025.

Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.

Sabri Eyuboglu, Ryan Ehrlich, Simran Arora, Neel Guha, Dylan Zinsley, Emily Liu, et al. Cartridges: Lightweight and general-purpose long context representations via self-study. *arXiv preprint arXiv:2506.06266*, 2025.

Xinyi Fan, Miri Liu, Ruozhen Yang, Siru Ouyang, and Jiawei Han. Can agent memory systems track evolving state? *arXiv preprint arXiv:2608.19652*, 2026.

Jizhan Fang, Xinle Deng, Haoming Xu, Ziyan Jiang, Yuqi Tang, Ziwen Xu, et al. LightMem: Lightweight and efficient memory-augmented generation. In *ICLR*, 2026. arXiv:2510.18866.

Zirui Guo, Lianghao Xia, Yanhua Yu, Tu Ao, and Chao Huang. LightRAG: Simple and fast retrieval-augmented generation. In *Findings of EMNLP 2025*, 2025.

Akshat Gupta, Anurag Rao, and Gopala Anumanchipalli. Model editing at scale leads to gradual and catastrophic forgetting. In *Findings of ACL 2024*, 2024.

Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. HippoRAG: Neurobiologically inspired long-term memory for large language models. In *NeurIPS*, 2024.

Bernal Jiménez Gutiérrez, Yiheng Shu, Weijian Qi, Sizhe Zhou, and Yu Su. From RAG to memory: Non-parametric continual learning for large language models. In *ICML*, 2025.

Zexue He, Yu Wang, Churan Zhi, Yuanzhe Hu, Tzu-Ping Chen, Lang Yin, Ze Chen, Tong Arthur Wu, Siru Ouyang, Zihan Wang, Jiaxin Pei, Julian McAuley, Yejin Choi, and Alex Pentland. MemoryArena: Benchmarking agent memory in interdependent multi-session agentic tasks. In *ICML*, 2026. arXiv:2602.16313.

Kelly Hong, Anton Troynikov, and Jeff Huber. Context rot: How increasing input tokens impacts LLM performance. Chroma technical report, 16 July 2025, 2025. Vendor research. https://www.trychroma.com/research/context-rot.

Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, Yang Zhang, and Boris Ginsburg. RULER: What's the real context size of your long-context language models? In *COLM*, 2024.

Chuanrui Hu, Xingze Gao, Zuyi Zhou, Dannong Xu, Yi Bai, Xintong Li, et al. EverMemOS: A self-organizing memory operating system for structured long-horizon reasoning. *arXiv preprint arXiv:2601.02163*, 2026a.

Yuanzhe Hu, Yu Wang, and Julian McAuley. Evaluating memory in LLM agents via incremental multi-turn interactions. In *ICLR*, 2026b. arXiv:2507.05257.

Yulin Hu, Zimo Long, Jiahe Guo, Xingyu Sui, Xing Fu, Weixiang Zhao, Yanyan Zhao, and Bing Qin. OP-Bench: Benchmarking over-personalization for memory-augmented personalized conversational agents. In *Proceedings of EMNLP 2026*, 2026c. arXiv:2601.13722.

Yuyang Hu, Shichun Liu, Yanwei Yue, Guibin Zhang, et al. Memory in the age of AI agents. *arXiv preprint arXiv:2512.13564*, 2025.

Jimin Huang, Yuyan Wang, Xueqing Peng, Sophia Ananiadou, and Junichi Tsujii. From memory to belief: A survey of state maintenance and belief revision in LLM decision agents. *SSRN preprint 7493158*, 2026a.

Wei-Chieh Huang, Weizhi Zhang, Yueqing Liang, Yuanchen Bei, Yankai Chen, Tao Feng, et al. A survey of agent memory in the second half: Towards self-evolving and long-horizon agents. *Transactions on Machine Learning Research*, 2026b. Survey Certification. arXiv:2602.06052.

Bowen Jiang, Yuan Yuan, Maohao Shen, Zhuoqun Hao, Zhangchen Xu, Zichen Chen, et al. PersonaMem-v2: Towards personalized intelligence via learning implicit user personas and agentic memory. *arXiv preprint arXiv:2512.06688*, 2025.

Dongming Jiang, Yi Li, Songtao Wei, Jinxin Yang, Ayushi Kishore, Alysa Zhao, Dingyi Kang, Xu Hu, Feng Chen, Qiannan Li, and Bingzhe Li. Anatomy of agentic memory: Taxonomy and empirical analysis of evaluation and system limitations. *arXiv preprint arXiv:2602.19320*, 2026.

Minki Kang, Wei-Ning Chen, Dongge Han, Huseyin A. Inan, Lukas Wutschitz, Yanzhi Chen, et al. ACON: Optimizing context compression for long-horizon LLM agents. In *ICML*, 2026. arXiv:2510.00615.

Andrej Karpathy. LLM wiki. GitHub gist, April 2026, 2026. https://gist.github.com/karpathy/442a6bf555914893e9891c11519de94f.

Ajay Kumar. Agent memory in 2026: Vendors claim 92.5 on LoCoMo, a plain filesystem scored

74, and every memory product lost to BM25. Blog post, 5 October 2026, 2026. Practitioner analysis. https://www.ajayk.sh/blog/agent-memory-systems-2026/.

Chingkwun Lam, Jiaxin Li, Lingfei Zhang, and Kuo Zhao. Governing evolving memory in LLM agents: Risks, mechanisms, and the stability and safety governed memory (SSGM) framework. *arXiv preprint arXiv:2603.11768*, 2026.

Chris Latimer, Nicoló Boschi, Andrew Neeser, Chris Bartholomew, Gaurav Srivastava, Xuan Wang, et al. Hindsight is 20/20: Building agent memory that retains, recalls, and reflects. *arXiv preprint arXiv:2512.12818*, 2025. Vendor research (Vectorize).

Letta. Benchmarking AI agent memory: Is a filesystem all you need? Letta research blog, 12 August 2025, 2025. Vendor research. https://www.letta.com/blog/benchmarking-ai-agent-memory/.

Xiaoyang Li, Yiqi Wang, Haohui Lu, Zhi Chen, Mo Li, Pingan Song, Mingkai Zheng, and Taotao Cai. MemTX: Transactional belief commit for stateful agent memory. *arXiv preprint arXiv:2607.23929*, 2026a.

Yifei Li, Weidong Guo, Lingling Zhang, Rongman Xu, Muye Huang, Hui Liu, et al. LoCoMo-Plus: Beyond-factual cognitive memory evaluation framework for LLM agents. *arXiv preprint arXiv:2602.10715*, 2026b.

Zhiyu Li, Chenyang Xi, Chunyu Li, Ding Chen, Boyu Chen, Shichao Song, et al. MemOS: A memory OS for AI system. *arXiv preprint arXiv:2507.03724*, 2025.

Junkai Lin, Ge Xu, Zheng Xu, Junkai Chen, and Xiangyuan Lan. Can agents truly forget? explore machine unlearning for LLM agents. *arXiv preprint arXiv:2610.17659*, 2026a.

Kevin Lin, Charlie Snell, Yu Wang, Charles Packer, Sarah Wooders, Ion Stoica, and Joseph E. Gonzalez. Sleep-time compute: Beyond inference scaling at test-time. *arXiv preprint arXiv:2504.13171*, 2025.

Zehao Lin, Xixuan Hao, Renyu Fu, Shaobo Cui, Kai Chen, Chunyu Li, Zhiyu Li, and Feiyu Xiong. A survey on long-term memory security in LLM agents: Attacks, defenses, and governance across the memory lifecycle. In *Proceedings of EMNLP 2026*, 2026b. arXiv:2604.16548.

Genglin Liu and Saadia Gabriel. PM-Bench: Evaluating prospective memory in LLM agents. In *COLM*, 2026. arXiv:2607.12385.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12, 2024.

Jinghao Luo, Yuchen Tian, Chuxue Cao, Ziyang Luo, Hongzhan Lin, Kaixin Li, Chuyi Kong, Ruichao Yang, and Jing Ma. From storage to experience: A survey on the evolution of LLM agent memory mechanisms. In *Findings of ACL 2026*, 2026. arXiv:2605.06716.

Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents. In *Proceedings of ACL 2024*, 2024.

Mem0. State of AI agent memory 2026: Benchmarks and trends. Mem0 blog, updated 10 October 2026, 2026a. Vendor publication. https://mem0.ai/blog/state-of-ai-agent-memory-2026.

Mem0. The state of agent wikis. Article, 21 July 2026, 2026b. Vendor publication. https://x.com/mem0ai/article/2079585032587694582.

Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in GPT. In *NeurIPS*, 2022.

Kevin Meng, Arnab Sen Sharma, Alex Andonian, Yonatan Belinkov, and David Bau. Mass-editing memory in a transformer. In *ICLR*, 2023.

Rui Miao, Yixin Liu, Yili Wang, Xu Shen, Yue Tan, Yiwei Dai, Shirui Pan, and Xin Wang. BlindGuard: Safeguarding LLM-based multi-agent systems under unknown attacks. In

Proceedings of ACL 2026, 2026. arXiv:2508.08127.

Haoliang Ming, Feifei Li, Xiaoqing Wu, and Wenhui Que. Retrieval as reasoning: Self-evolving agent-native retrieval via LLM-Wiki. *arXiv preprint arXiv:2605.25480*, 2026.

Ali Modarressi, Hanieh Deilamsalehy, Franck Dernoncourt, Trung Bui, Ryan A. Rossi, Seunghyun Yoon, and Hinrich Schütze. NoLiMa: Long-context evaluation beyond literal matching. In *ICML*, 2025.

Tsendsuren Munkhdalai, Manaal Faruqui, and Siddharth Gopal. Leave no context behind: Efficient infinite context transformers with infini-attention. *arXiv preprint arXiv:2404.07143*, 2024.

Thong Q. Nguyen, Shubhang Desai, Raja Hasnain Anwar, Firoz Shaik, Vishwas Suryanarayanan, and Vishal Chowdhary. VerificAgent: Domain-specific memory verification for scalable oversight of aligned computer-use agents. *arXiv preprint arXiv:2506.02539*, 2025.

Jingwei Ni, Yihao Liu, Xinpeng Liu, Yutao Sun, Mengyu Zhou, Pengyu Cheng, et al. Trace2Skill: Distill trajectory-local lessons into transferable agent skills. *arXiv preprint arXiv:2603.25158*, 2026.

Yasmine Omri, Ziyu Gan, Zachary Broveak, Robin Geens, Zexue He, Alex Pentland, Marian Verhelst, Tsachy Weissman, and Thierry Tambe. Agent memory: Characterization and system implications of stateful long-horizon workloads. *arXiv preprint arXiv:2606.06448*, 2026.

OpenAI. Memory in ChatGPT. Help Center article, 2026. Vendor documentation, accessed 25 October 2026. https://help.openai.com/en/articles/8590148-memory-faq.

Siru Ouyang, Jun Yan, I-Hung Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, et al. ReasoningBank: Scaling agent self-evolving with reasoning memory. In *ICLR*, 2026. arXiv:2509.25140.

Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

Nils Palumbo, Sarthak Choudhary, Jihye Choi, Guy Amir, Prasad Chalasani, Isil Dillig, and Somesh Jha. Formal policy enforcement for agentic systems. *arXiv preprint arXiv:2602.16708*, 2026.

Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *UIST*, 2023.

Young Bin Park. Graph-native cognitive memory for AI agents: Formal belief revision semantics for versioned memory architectures. *arXiv preprint arXiv:2603.17244*, 2026.

Mathis Pink, Qinyuan Wu, Vy Ai Vo, Javier Turek, Jianing Mu, Alexander Huth, and Mariya Toneva. Position: Episodic memory is the missing piece for long-term LLM agents. *arXiv preprint arXiv:2502.06975*, 2025.

Jack W. Rae, Anna Potapenko, Siddhant M. Jayakumar, and Timothy P. Lillicrap. Compressive transformers for long-range sequence modelling. In *ICLR*, 2020.

Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. Zep: A temporal knowledge graph architecture for agent memory. *arXiv preprint arXiv:2501.13956*, 2025. Vendor research.

Santhosh Kumar Ravindran. Portable agent memory: A protocol for cryptographically-verified memory transfer across heterogeneous AI agents. *arXiv preprint arXiv:2605.11032*, 2026.

Alireza Rezazadeh, Zichao Li, Ange Lou, Yuying Zhao, Wei Wei, and Yujia Bao. Collaborative memory: Multi-user memory sharing in LLM agents with dynamic access control. *arXiv preprint arXiv:2505.18279*, 2025.

Tianneng Shi, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, and Dawn Song. Progent: Securing AI agents with privilege control. *arXiv preprint arXiv:2504.11703*, 2025.

Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *NeurIPS*, 2023.

Xiaonan Si, Meilin Zhu, Simeng Qin, Lijia Yu, Lijun Zhang, Shuaitong Liu, et al. SeCon-RAG: A two-stage semantic filtering and conflict-free framework for trustworthy RAG. In *NeurIPS*, 2025. arXiv:2510.09710.

Richard T. Snodgrass. *Developing Time-Oriented Database Applications in SQL*. Morgan Kaufmann, 1999.

Quentin Spencer. Ground truth first: A longitudinal evaluation instrument for agent memory, and the tenure crossover in memory-architecture rankings. *arXiv preprint arXiv:2607.21962*, 2026.

Mujahid Sultan, Sri Thuraisamy, and Daya Rajaratnam. NeuSymMS: A hybrid neuro-symbolic memory system for persistent, self-curating LLM agents. *arXiv preprint arXiv:2605.17596*, 2026.

Theodore R. Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas L. Griffiths. Cognitive architectures for language agents. *Transactions on Machine Learning Research*, 2024. arXiv:2309.02427.

Yu Sun, Xinhao Li, Karan Dalal, Jiarui Xu, Arjun Vikram, Genghan Zhang, et al. Learning to (learn at test time): RNNs with expressive hidden states. *arXiv preprint arXiv:2407.04620*, 2024.

Balachandra Devarangadi Sunil, Isheeta Sinha, Piyush Maheshwari, Shantanu Todmal, Shreyan Mallik, and Shuchi Mishra. Memory poisoning attack and defense on memory based LLM-agents. *arXiv preprint arXiv:2601.05504*, 2026.

Mirac Suzgun, Mert Yuksekgonul, Federico Bianchi, Dan Jurafsky, and James Zou. Dynamic cheatsheet: Test-time learning with adaptive memory. *arXiv preprint arXiv:2504.07952*, 2025.

Xue Tan, Hao Luan, Mingyu Luo, Xiaoyan Sun, Ping Chen, and Jun Dai. RevPRAG: Revealing poisoning attacks in retrieval-augmented generation through LLM activation analysis. In *Findings of EMNLP 2025*, 2025. arXiv:2411.18948.

Mohammad Tavakoli, Alireza Salemi, Carrie Ye, Mohamed Abdalla, Hamed Zamani, and J. Ross Mitchell. Beyond a million tokens: Benchmarking and enhancing long-term memory in LLMs. *arXiv preprint arXiv:2510.27246*, 2025.

Bin Wang, Fan Wang, Pingping Wang, Jinyu Cong, Yang Yu, Yilong Yin, et al. Agentic unlearning: When LLM agent meets machine unlearning. *arXiv preprint arXiv:2602.17692*, 2026a.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *Transactions on Machine Learning Research*, 2024a.

Yiqi Wang, Jiaqi Zhang, Zhangkai Wu, Taotao Cai, Zirui Liu, Zequn Sun, et al. From agent traces to trust: A survey of evidence tracing and execution provenance in LLM agents. *arXiv preprint arXiv:2606.04990*, 2026b.

Yu Wang and Xi Chen. MIRIX: Multi-agent memory system for LLM-based agents. *arXiv preprint arXiv:2507.07957*, 2025.

Yu Wang, Yifan Gao, Xiusi Chen, Haoming Jiang, Shiyang Li, Jingfeng Yang, et al. MemoryLLM: Towards self-updatable large language models. In *ICML*, 2024b.

Yu Wang, Dmitry Krotov, Yuanzhe Hu, Yifan Gao, Wangchunshu Zhou, Julian McAuley, Dan Gutfreund, Rogerio Feris, and Zexue He. M+: Extending MemoryLLM with scalable long-term memory. In *ICML*, 2025a.

Yu Wang, Ryuichi Takanobu, Zhiqi Liang, Yuzhen Mao, Yuanzhe Hu, Julian McAuley, and Xiaojian Wu. Mem-α: Learning memory construction via reinforcement learning. *arXiv*

preprint arXiv:2509.25911, 2025b.

Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent workflow memory. *arXiv preprint arXiv:2409.07429*, 2024c.

Qianshan Wei, Tengchao Yang, Yaochen Wang, Xinfeng Li, Lijun Li, Zhenfei Yin, et al. A-MemGuard: A proactive defense framework for LLM-based agent memory. In *ICML*, 2026. arXiv:2510.02373, 2025.

Sarah Wooders. Why memory isn't a plugin. Letta blog, 3 April 2026, 2026. Vendor publication. https://www.letta.com/blog/why-memory-isnt-a-plugin/.

Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. LongMemEval: Benchmarking chat assistants on long-term interactive memory. In *ICLR*, 2025a.

Tingyu Wu, Zhisheng Chen, Ziyan Weng, Shuhe Wang, Chenglong Li, Shuo Zhang, et al. KnowMe-Bench: Benchmarking person understanding for lifelong digital companions. *arXiv preprint arXiv:2601.04745*, 2026a.

Yaxiong Wu, Sheng Liang, Chen Zhang, Yichao Wang, Yongyue Zhang, Huifeng Guo, Ruiming Tang, and Yong Liu. From human memory to AI memory: A survey on memory mechanisms in the era of LLMs. *arXiv preprint arXiv:2504.15965*, 2025b.

Yifan Wu, Lizhu Zhang, Yuhang Zhou, Mingyi Wang, Bo Peng, Serena Li, Xiangjun Fan, and Zhuokai Zhao. Remember when it matters: Proactive memory agent for long-horizon agents. *arXiv preprint arXiv:2607.08716*, 2026b.

Yuhao Wu, Franziska Roesner, Tadayoshi Kohno, Ning Zhang, and Umar Iqbal. IsolateGPT: An execution isolation architecture for LLM-based agentic systems. In *Network and Distributed System Security Symposium (NDSS)*, 2025c. arXiv:2403.04960.

Yuhuai Wu, Markus N. Rabe, DeLesley Hutchins, and Christian Szegedy. Memorizing transformers. In *ICLR*, 2022.

Chong Xiang, Tong Wu, Zexuan Zhong, David Wagner, Danqi Chen, and Prateek Mittal. Certifiably robust RAG against retrieval corruption. In *IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*, 2026. arXiv:2405.15556, 2024.

Bofan Xie, Chaokun Wang, Guanchen Ge, Songyao Wang, Fang Niu, Hang Zhang, et al. Can agents forget when asked? a benchmark for memory unlearning in LLM agents. *arXiv preprint arXiv:2610.18052*, 2026.

Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-MEM: Agentic memory for LLM agents. In *NeurIPS*, 2025. arXiv:2502.12110.

Xiaoyu Xu, Minxin Du, Li Bai, Junxu Liu, Yaxin Xiao, Kun Fang, et al. Machine unlearning for large language models: Foundations, advances, and agentic extensions. *arXiv preprint arXiv:2609.30909*, 2026a.

Yue Xu, Yutao Sun, Yihao Liu, Mengyu Zhou, Jiayi Qiao, Lu Ma, et al. From passive retrieval to active memory navigation: Learning to use memory as a structured action space. *arXiv preprint arXiv:2607.05794*, 2026b.

Neeraj Yadav. Temporal validity in retrieval memory: Eliminating stale-fact errors for AI agents over evolving knowledge. *arXiv preprint arXiv:2606.26511*, 2026.

Sikuan Yan, Xiufeng Yang, Zuchao Huang, Ercong Nie, Zifeng Ding, Zonggen Li, et al. Memory-R1: Enhancing large language model agents to manage and utilize memories via reinforcement learning. In *Proceedings of ACL 2026*, 2026. arXiv:2508.19828.

Hongkang Yang, Zehao Lin, Wenjin Wang, Hao Wu, Zhiyu Li, Bo Tang, et al. Memory^3: Language modeling with explicit memory. *Journal of Machine Learning*, 3:300–346, 2024. arXiv:2407.01178.

Yanyu Yao, Shangze Li, Zhi Zheng, Hui Zheng, Qi Liu, Tong Xu, and Enhong Chen. AtomMem: Building simple and effective memory system for LLM agents via atomic facts. *arXiv preprint arXiv:2606.19847*, 2026.

Hongli Yu, Tinghong Chen, Jiangtao Feng, Jiangjie Chen, Weinan Dai, Qiying Yu, et al. MemAgent: Reshaping long-context LLM with multi-conv RL-based memory agent. In *ICLR*, 2026a. Oral. arXiv:2507.02259.

Yi Yu, Liuyi Yao, Yuexiang Xie, Qingquan Tan, Jiaqi Feng, Yaliang Li, and Libing Wu. Agentic memory: Learning unified long-term and short-term memory management for large language model agents. In *Proceedings of ACL 2026*, 2026b. SAC Highlight. arXiv:2601.01885.

Zep. Lies, damn lies, and statistics: Is Mem0 really SOTA in agent memory? Zep blog, 6 May 2025, 2025. Vendor publication. https://www.getzep.com/blog/lies-damn-lies-statistics-is-mem0-really-sota-in-agent-memory/.

Baolei Zhang, Haoran Xin, Minghong Fang, Zhuqing Liu, Biao Yi, Tong Li, and Zheli Liu. Traceback of poisoning attacks to retrieval-augmented generation. In *Proceedings of The Web Conference (WWW)*, 2025a. arXiv:2504.21668.

Guibin Zhang, Muxin Fu, Guancheng Wan, Miao Yu, Kun Wang, and Shuicheng Yan. G-Memory: Tracing hierarchical memory for multi-agent systems. *arXiv preprint arXiv:2506.07398*, 2025b.

Ningning Zhang, Xingxing Yang, Zhizhong Tan, Weiping Deng, and Wenyong Wang. HiMem: Hierarchical long-term memory for LLM long-horizon agents. *arXiv preprint arXiv:2601.06377*, 2026a.

Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, et al. Agentic context engineering: Evolving contexts for self-improving language models. In *ICLR*, 2026b. arXiv:2510.04618.

Zeyu Zhang, Xiaohe Bo, Chen Ma, Rui Li, Xu Chen, Quanyu Dai, Jieming Zhu, Zhenhua Dong, and Ji-Rong Wen. A survey on the memory mechanism of large language model based agents. *arXiv preprint arXiv:2404.13501*, 2024. Later published by ACM, 2025.

Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. ExpeL: LLM agents are experiential learners. In *AAAI*, 2024.

Yujie Zhao, Boqin Yuan, Junbo Huang, Haocheng Yuan, Zhongming Yu, Haozhou Xu, Lanxiang Hu, Abhilash Shankarampeta, Zimeng Huang, Wentao Ni, Yuandong Tian, and Jishen Zhao. AMA-Bench: Evaluating long-horizon memory for agentic applications. *arXiv preprint arXiv:2602.22769*, 2026.

Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. MemoryBank: Enhancing large language models with long-term memory. In *AAAI*, 2024.

Huichi Zhou, Yihang Chen, Siyuan Guo, Xue Yan, Kin Hei Lee, Zihan Wang, et al. Memento: Fine-tuning LLM agents without fine-tuning LLMs. *arXiv preprint arXiv:2508.16153*, 2025a.

Huichi Zhou, Kin-Hei Lee, Zhonghao Zhan, Yue Chen, Zhenhao Li, Zhaoyang Wang, Hamed Haddadi, and Emine Yilmaz. TrustRAG: Enhancing robustness and trustworthiness in retrieval-augmented generation. *arXiv preprint arXiv:2501.00879*, 2025b.

Zijian Zhou, Ao Qu, Zhaoxuan Wu, Sunghwan Kim, Alok Prakash, Daniela Rus, et al. MEM1: Learning to synergize memory and reasoning for efficient long-horizon agents. In *ICLR*, 2026. arXiv:2506.15841.

Qiming Zhu, Shunian Chen, Rui Yu, Zhehao Wu, and Benyou Wang. From lossy to verified: A provenance-aware tiered memory for agents. *arXiv preprint arXiv:2602.17913*, 2026.

A Search Protocol

Sources and date. Searches were run on 25 October 2026 against arXiv (via its listing, abstract, and API endpoints), alphaXiv paper overviews and "similar papers" lists, the proceedings of NeurIPS, ICML, ICLR, ACL, EMNLP, COLM, TMLR, and AAAI, and vendor and practitioner publications (engineering blogs, product documentation, GitHub repositories). alphaXiv was

used for structured overviews of paper content; author lists and venues were taken from arXiv abstract pages or the arXiv API, because alphaXiv renders author lists client-side.

Queries. Seed queries combined {agent memory, LLM memory, long-term memory, episodic memory, memory system} with {survey, benchmark, evaluation, security, poisoning, temporal, staleness, provenance, unlearning, deletion, multi-agent, personalization, reinforcement learning, compaction, context engineering, skills, coding agents, prospective memory, calibration, belief revision}. Each seed hit was snowballed forward through alphaXiv "similar papers" and backward through its references, with the eight surveys of Table 2 as the main backward-snowballing roots.

Inclusion. A work was included if it (i) proposes, evaluates, attacks, or governs a mechanism by which an LLM-based agent carries information across steps or sessions, or (ii) is a benchmark or methodological analysis of such mechanisms, or (iii) is a vendor or practitioner report describing shipped behaviour or reporting measurements of such a system. Works on long-context modelling, static-corpus RAG, and continual pretraining were included only where cited by the agent-memory literature as baselines or ingredients. Pure position papers were included where later work cites their definitions.

Counts and grading. The reference list contains 134 works: 53 with a peer-reviewed venue recorded, 63 arXiv or SSRN preprints, and 18 vendor, practitioner, or documentation sources. Grading was applied per work; where a preprint was later accepted, the venue is recorded in the reference list and the text treats it as peer-reviewed.

Limitations of the protocol. The search was conducted by a single reviewer with AI assistance over one day; it is not a systematic review with dual screening. Numbers are taken from abstracts and alphaXiv overviews and were not independently reproduced. Long author lists are abbreviated. Several of the strongest results are preprints and may change on revision.

B Representative Systems by Family

Tables 8 and 9 characterise the research systems discussed in the body along the review's three axes and records the evaluation each reports. It complements Table 5, which covers frameworks and products.

Table 8: Representative systems (part 1: parametric, context-resident, and retrieval families). Substrate: P = parametric, L = latent, T = token-level. Function: W = working, E = episodic, S = semantic, Pr = procedural. Control: pipe = fixed extraction pipeline; agent = agent-directed via tools; sep = separate memory agent; RL = learned policy; human = human-written. Evidence: PR = peer-reviewed venue; pre = preprint; vend = vendor paper.

System	Substr.	Func.	Control	Reported evaluation	Evid.
Parametric and latent (Section 3)					
Titans (Behrouz et al., 2024)	L	W/E	–	Needle-in-haystack beyond 2M tokens	pre
Memory Layers (Berges et al., 2024)	P	S	–	Factual QA at matched compute	pre
Cartridges (Eyuboglu et al., 2025)	L	S	offline	38.6× memory, 26.4× throughput vs. in-context	pre
M+ (Wang et al., 2025a)	L	S/E	–	Retention beyond 160k tokens	PR
Context-resident and compaction (Section 4)					
MemGPT (Packer et al., 2023)	T	W/E/S	agent	DMR; document QA	pre
ACE (Zhang et al., 2026b)	T	Pr	agent	+10.6% agent, +8.6% finance	PR
ACON (Kang et al., 2026)	T	W	pipe	26–54% token reduction, AppWorld, OfficeBench	PR
TierMem (Zhu et al., 2026)	T	E/S	pipe	LoCoMo 0.851 vs. 0.873 raw at 54% fewer tokens	pre
MEM1 (Zhou et al., 2026)	T	W	RL	3.5× memory reduction, 16-objective tasks	PR
MemAgent (Yu et al., 2026a)	T	W	RL	Long-document QA beyond window	PR
External retrieval stores (Section 5)					
Mem0 (Chhikara et al., 2025)	T	S	pipe	LoCoMo 66.9 (author-run; disputed)	vend
Zep (Rasmussen et al., 2025)	T	E/S	pipe	LongMemEval 71.2 vs. 60.2 full context	vend
HippoRAG 2 (Gutiérrez et al., 2025)	T	S	pipe	Multi-hop and associative QA	PR
A-MEM (Xu et al., 2025)	T	E/S	pipe	LoCoMo; DialSim	PR
Hindsight (Latimer et al., 2025)	T	E/S	pipe	LoCoMo (author-run); LongMemEval	vend
MIRIX (Wang and Chen, 2025)	T	all	sep	ScreenshotVQA; LoCoMo (author-run)	pre
LightMem (Fang et al., 2026)	T	E/S	offline	Large token and call reductions vs. Mem0-style	PR
Kumiho (Park, 2026)	T	E/S	pipe	LoCoMo-Plus, independently reproduced mid-80s	pre
LLM-Wiki (Ming et al., 2026)	T	S	agent	+2.0–8.1 F1 over HippoRAG 2, LightRAG, GraphRAG	pre
Letta filesystem (Letta, 2025)	T	E	agent	LoCoMo 74.0	vend

Table 9: Representative systems, continued: experiential, learned-policy, personalisation, and multi-agent systems. Abbreviations as in Table 8.

System	Substr.	Func.	Control	Reported evaluation	Evid.
Experiential and procedural (Section 6)					
Reflexion (Shinn et al., 2023)	T	Pr	agent	HumanEval pass@1 91%	PR
ExpeL (Zhao et al., 2024)	T	Pr	agent	Cross-task insights, HotpotQA, ALFWorld	PR
AWM (Wang et al., 2024c)	T	Pr	agent	WebArena +24.6%, Mind2Web +51.1% relative	pre
ReasoningBank (Ouyang et al., 2026)	T	Pr	agent	Up to 34.2% relative, WebArena, SWE-Bench-Verified	PR
Memento (Zhou et al., 2025a)	T	E	RL	GAIA validation 87.9%	pre
Trace2Skill (Ni et al., 2026)	T	Pr	offline	Cross-model transfer; +57.65 pp WikiTableQuestions	pre
REM (Chen et al., 2026)	T	E/Pr	gated	SWE-ContextBench Lite 27.6 vs. 24.9 resolution	PR (wksp)
Learned management policies (Section 7)					
Memory-R1 (Yan et al., 2026)	T	S	RL	LoCoMo SOTA from 152 training pairs	PR
Mem-α (Wang et al., 2025b)	T	W/E/S	RL	30k-token training to 400k-token evaluation	pre
AgeMem (Yu et al., 2026b)	T	W/E/S	RL	Unified long/short-term management	PR
Proactive Memory Agent (Wu et al., 2026b)	T	W	sep/RL	Terminal-Bench 2.0 +8.3 pp, τ^2-Bench +6.8 pp	pre
NapMem (Xu et al., 2026b)	T	E/S	RL	PersonaMem-v2, LongMemEval, LoCoMo	pre
MemCalib-RL (Cao et al., 2026)	T	use	RL	Balanced over/under-use across three model families	pre
Personalisation and multi-agent (Sections 8, 9)					
EverMemOS (Hu et al., 2026a)	T	E/S	pipe	LoCoMo, LongMemEval (author-run)	pre
AtomMem (Yao et al., 2026)	T	S	pipe	LoCoMo (author-run)	pre
G-Memory (Zhang et al., 2025b)	T	E/S	sep	Multi-agent embodied and QA tasks	pre
MemTX (Li et al., 2026a)	T	S	gated	Property tests over 5.5M protocol states	pre